

SOFTWARE QUALITY ASSURANCE PLAN

Multimedia gallery steganography detection safe

V1.0

20/07/2026

Grupo 4

UNIVERSIDAD DE LAS FUERZAS ARMADAS ESPE

**Av. Gral. Rumiñahui S/N, 171103 Sangolquí
Departamento de Ciencias de la Computación**

Aprobado para distribución pública; distribución ilimitada

SOFTWARE QUALITY ASSURANCE PLAN

Multimedia gallery steganography detection safe

V1.0

20/07/2026

SQA Plan Approvals:

_____	<u>22/07/2026</u>
SQA Manager	Date
_____	<u>22/07/2026</u>
Project Manager	Date
_____	<u>22/07/2026</u>
Program Manager	Date

PREFACIO

Este documento contiene el Plan de Aseguramiento de la Calidad del Software (SQA) para el proyecto *Multimedia gallery steganography detection safe*. Las actividades de SQA descritas en este plan son consistentes con el Plan de Desarrollo de Software (o Plan de Gestión del Proyecto) y otros documentos de planificación del proyecto. Este documento ha sido adaptado de la Plantilla del Plan SQA, TM-SQA-01, v2.0.

La [Código/Proyecto/Oficina] asume la responsabilidad de este documento y lo actualiza, según sea necesario, para satisfacer las necesidades del proyecto. Los usuarios de este documento pueden reportar deficiencias o correcciones utilizando la Solicitud de Cambio de Documento que se encuentra al final del documento. Las actualizaciones de este documento se realizarán, al menos anualmente, de acuerdo con el [Proceso de Gestión de Configuración del Proyecto].

REGISTRO DE CAMBIOS

A - AÑADIDO

M - MODIFICADO

D - ELIMINADO

Versión	Fecha	Número de Figura, Tabla o Párrafo	A/M/D	Título o Descripción Breve	Solicitud de Cambio
1.0	17/07/26	Documento Completo		Generación del documento inicial del sistema	

Índice

1. SECCIÓN 1. PROPÓSITO	9
1.1. 1.1 ALCANCE	9
1.2. 1.2 VISIÓN GENERAL DEL SISTEMA	9
1.3. 1.3 VISTA GENERAL DEL DOCUMENTO	10
1.4. 1.6 DOCUMENTOS DE REFERENCIA	11
2. SECCIÓN 2. GESTIÓN	14
2.1. 2.1 ORGANIZACIÓN	14
2.2. 2.2 RECURSOS	17
2.2.1. Instalaciones y Equipos	17
2.2.2. Personal	17
2.2.3. Calificaciones del Personal	18
3. SECCIÓN 3. TAREAS DE SQA	20
3.1. 3.1 TAREA: REVISIÓN DE PRODUCTOS DE SOFTWARE	20
3.2. 3.2 TAREA: REVISIÓN DE HERRAMIENTAS DE SOFTWARE	20
3.3. 3.3 TAREA: EVALUAR PROCESO DE REVISIÓN DE PRODUCTOS DE SOFTWARE	20
3.4. 3.4 TAREA: EVALUAR PROCESOS DE PLANIFICACIÓN, MONITOREO Y SUPERVISIÓN DE PROYECTOS	20
3.5. 3.5 TAREA: EVALUAR PROCESOS DE IMPLEMENTACIÓN, PRUEBAS UNITARIAS E INTEGRACIÓN	21
3.6. 3.6 TAREA: EVALUAR PROCESOS DE CONTROL DE CÓDIGO	22
3.7. 3.7 TAREA: VERIFICAR REVISIONES DE PROCESOS Y AUDITORÍAS	23
3.7.1. Tarea: Verificar Revisiones Técnicas	23
3.7.2. Tarea: Verificar Revisiones de Gestión	24
3.7.3. Tarea: Conducir Auditorías de Procesos	24
3.7.4. Tarea: Realizar Auditorías de Configuración	24
3.8. 3.8 RESPONSABILIDADES	25
3.9. 3.9 CRONOGRAMA	27
4. SECCIÓN 4. DOCUMENTACIÓN	29
5. SECCIÓN 5. ESTÁNDARES, PRÁCTICAS, CONVENCIONES Y MÉTRICAS	32
5.1. 5.1 MÉTRICAS	32
5.2. 5.2 MÉTRICAS DE CALIDAD EN USO	33
5.2.1. Efectividad	33
5.2.2. Eficiencia	34
5.2.3. Satisfacción	35
5.2.4. Libertad de Riesgo	35
5.2.5. Cobertura de Contexto	36
5.3. Métricas de Calidad de Producto	36

5.3.1. Eficiencia de rendimiento	37
5.3.2. Seguridad	37
5.3.3. Mantenibilidad	37
6. SECCIÓN 6. TESTS	40
6.1. 6.1 SISTEMA BAJO PRUEBA Y NIVELES DE PRUEBA	42
6.2. 6.2 MATRIZ DE TRAZABILIDAD	42
6.3. 6.3 CASOS DE PRUEBA	44
6.3.1. 6.3.1 RF-01: Registro y Autenticación Segura	44
6.3.2. 6.3.2 RF-02: Control de Acceso y Sesión	46
6.3.3. 6.3.3 RF-03: Detección de Esteganografía	48
6.3.4. 6.3.4 RF-04: Cuarentena y Revisión Manual	51
6.3.5. 6.3.5 RF-05: Panel de Supervisor, Auditoría y Protección XSS	56
6.4. 6.4 RESUMEN DE EJECUCIÓN	61
6.5. 6.5 HALLAZGOS DERIVADOS DE LA EJECUCIÓN	61
7. SECCIÓN 7. REPORTES DE PROBLEMAS SQA Y RESOLUCIONES	64
7.1. 7.1 Proceso de reporte de auditoría	64
7.1.1. 7.1.1 Reporte de hallazgos	64
7.1.2. 7.1.2 Relación de los hallazgos con las métricas de calidad	65
7.2. 7.2 Registro de problemas en el software	65
7.3. 7.3 Informe de evaluación de herramientas de software	66
7.4. Trazabilidad de resultados a métricas de calidad	67
8. SECCIÓN 8. HERRAMIENTAS, TÉCNICAS Y METODOLOGÍAS	71
9. SECCIÓN 9. CONTROL DEL CÓDIGO	73
10. SECCIÓN 10. RECOPIACIÓN DE EVIDENCIAS, MANTENIMIENTO Y RETENCIÓN	76
10.1. Evidencias fotográficas	76
11. SECCIÓN 11. PREPARACIÓN DEL PERSONAL	79
12. SECCIÓN 12. GESTIÓN DE RIESGOS	82
A. APÉNDICE A. LISTA DE ACRÓNIMOS	86
B. APÉNDICE B. LISTAS DE VERIFICACIÓN DE AUDITORÍA DE PROCESOS DE INGENIERÍA DE SOFTWARE GENERALES	88

SECCIÓN 1. PROPÓSITO

El presente Plan de Aseguramiento de la Calidad está orientado a una Galería de Imágenes Segura desarrollada con Spring Boot, React y PostgreSQL, cuya principal característica es la detección de contenido oculto mediante técnicas de análisis de esteganografía. Con este documento se establece el alcance, la estrategia de las pruebas, los criterios de aceptación, las herramientas a utilizarse y la gestión de riesgos asociados al sistema, con el propósito de garantizar su correcto funcionamiento, fortalecer su nivel de seguridad y asegurar una experiencia confiable para los usuarios finales a través de un proceso estructurado de validación y mejora continua.

1.1 ALCANCE

El sistema bajo prueba (SUT) es una Galería de Imágenes Segura (backend Spring Boot 4.0.6 / Java 21 + frontend React 18 + Vite, persistencia en PostgreSQL). Sus principales módulos a probar son: Gestión de autenticación de usuarios, carga y almacenamiento de imágenes, organización en álbumes, detección de esteganografía basada en análisis de anomalías LSB (bits menos significativos) y metadatos EXIF, cuarentena de archivos sospechosos, y un panel de supervisor para auditoría.

Se realizará una identificación de puntos clave del sistema, herramientas y definición de mediciones para la correcta evaluación de las funcionalidades. Finalmente, se presentará un reporte de los hallazgos y posibles mejoras dentro del sistema.

1.2 VISIÓN GENERAL DEL SISTEMA

La Galería de Imágenes Segura es una aplicación web que permite a los usuarios subir, organizar y compartir imágenes en álbumes, con la característica diferenciadora de detectar contenido oculto mediante análisis de esteganografía (anomalías EOF, firmas embebidas y metadatos EXIF). El sistema incluye 4 subsistemas principales: autenticación y autorización, gestión de imágenes y álbumes, motor de detección de esteganografía con cuarentena, y panel de supervisor para auditoría. La Figura 1-1 identifica los CI dentro de cada subsistema y destaca aquellos a los que se aplica este Plan SQA.

Cuadro 2: Actividades del Ciclo de Vida del Software

ACTIVIDAD DEL CICLO DE VIDA DEL SOFTWARE
Planificación y Supervisión del Proyecto
Entorno de Desarrollo de Software
Análisis de Requisitos del Sistema
Diseño del Sistema
Análisis de Requisitos del Software
Diseño del Software
Implementación del Software y Pruebas Unitarias
Pruebas de Integración de Unidades
Pruebas de Calificación de CI
Pruebas de Integración CI/CIHW y del Sistema
Pruebas de Calificación del Sistema
Preparación para el Uso del Software
Preparación para la Transición del Software
Mantenimiento del Ciclo de Vida

Cuadro 3: Nomenclatura/Identificación de CI

NOMENCLATURA	ACRÓNIMO	NÚMERO DE CI
Plan de Aseguramiento de Calidad	SQAP	CI-01
Plan de Pruebas de Software	STP	CI-04
Reporte de Pruebas de Software	STR	CI-05
Documentación de Usuario	UDP	CI-06
Guía de Despliegue	DEP	CI-07

1.3 VISTA GENERAL DEL DOCUMENTO

Este documento identifica las organizaciones y procedimientos que se utilizarán para realizar las actividades relacionadas con el programa SQA del proyecto, según lo especificado en el estándar IEEE Std 730-1998, *IEEE Standard for Software Quality Assurance Plans*, referencia (c), y IEEE Std 730.1-1995, *IEEE Guide for SQA Planning*, referencia (d).

La Sección 1 identifica el sistema al que se aplica este Plan SQA; proporciona una visión general del sistema y sus funciones de software; resume el propósito y el contenido del Plan SQA; describe la relación del Plan SQA con otros planes de gestión y enumera todos los documentos referenciados en este Plan SQA.

La Sección 2 describe cada elemento principal de la organización que influye en la calidad del software.

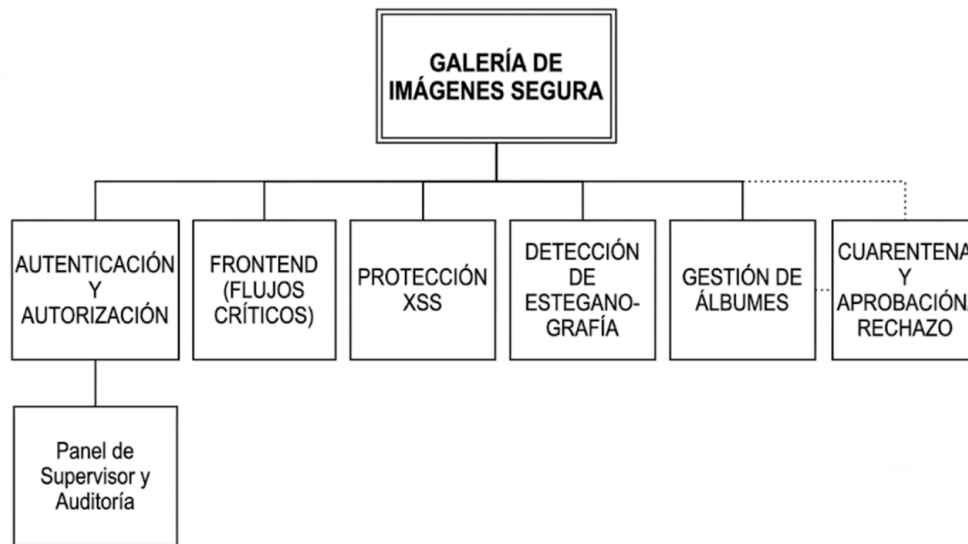


Figura 1: Software de Galería de Imágenes Segura

La Sección 3 describe las diversas tareas de SQA. La Sección 4 enumera los documentos base producidos y mantenidos por el proyecto. La Sección 5 identifica los estándares, prácticas y convenciones. La Sección 6 describe la participación de SQA en las pruebas. La Sección 7 describe el reporte de problemas y la acción correctiva. La Sección 8 describe las herramientas, técnicas y metodologías de SQA. La Sección 9 describe la herramienta de gestión de configuración utilizada para el control del código. La Sección 10 describe la evaluación de SQA del control de medios. La Sección 11 describe el control del software de proveedores. La Sección 12 describe los procedimientos de SQA para la recopilación, mantenimiento y retención de registros. La Sección 13 describe los requisitos de capacitación de SQA. La Sección 12 describe la revisión de SQA del proceso de Gestión de Riesgos. El Apéndice A proporciona una lista de acrónimos. El Apéndice B proporciona listas de verificación que se utilizarán o adaptarán para verificar el cumplimiento de las mejores prácticas generales de ingeniería de software.

1.6 DOCUMENTOS DE REFERENCIA

1. SSC San Diego Software Quality Assurance Policy, Version 1.1, October 1997.
2. IEEE/EIA Standard 12207 Series - Standard for Information Technology – Software life cycle processes, March 1998.
3. IEEE-Std-730-1998, IEEE Standard for Software Quality Assurance Plans, June 1998.
4. IEEE-Std-730.1-1995, IEEE Guide for Software Quality Assurance Planning, December 1995.
5. Multimedia gallery steganography detection safe Software Development Plan, [Identificación del Documento], [Fecha del Documento].

6. Multimedia gallery steganography detection safe Software Configuration Management Plan, [Identificación del Documento], [Fecha del Documento].
7. Software Engineering Process Policy, SPAWARSYSCEN SAN DIEGO INST 5234.1, July 2000.
8. Multimedia gallery steganography detection safe Program Plan, [Identificación del Documento], [Fecha del Documento].
9. Software Quality Assurance Process, PR-SQA-02.
10. Software Quality Assurance Plan Template, TM-SQA-01.
11. A Description of the Space and Naval Warfare System Center San Diego Software Process Assets, PR-OPD-03.
12. MIL-STD-1521, Technical Reviews and Audits for Systems, Equipments, and Computer Software.
13. MIL-STD-973, Configuration Management. NOTA: Esta norma ha sido reemplazada por EIA-649, el Estándar de Consenso para la Gestión de Configuración, pero las disposiciones de MIL-STD-973 se consideran útiles como guía para desarrollar actividades de Gestión de Configuración de Software.
14. Peer Review Process, PR-PR-02.
15. Military Standard (MIL-STD)-498, Software Development and Documentation, Data Item Descriptions (DIDs). NOTA: Aunque MIL-STD-498 ha sido reemplazado por IEEE Std 12207, los DIDs para MIL-STD-498 todavía se consideran aplicables para el apoyo al desarrollo de procedimientos de ingeniería de software y documentación de soporte.
16. IEEE Std 1045-1992, IEEE Standard for Software Productivity Metrics, September 1992.
17. IEEE Std 1061-1992, IEEE Standard for a Software Quality Metrics Methodology, December 1992.
18. IEEE Std 982.1-1988, IEEE Standard Dictionary of Measures to Produce Reliable Software, June 1988.
19. Std 982.2-1988, IEEE Guide for the Use of IEEE Standard Dictionary of Measures to Produce Reliable Software, September 1988.

SECCIÓN 2. GESTIÓN

Esta sección describe cada elemento principal de la organización que influye en la calidad del software del proyecto Galería de Imágenes Segura con Detección de Esteganografía.

2.1 ORGANIZACIÓN

La buena práctica del software requiere un cierto grado de independencia para el grupo SQA. Esta independencia proporciona una fortaleza clave a SQA; es decir, SQA tiene la libertad, si la calidad del producto está en peligro, de informar esta posibilidad directamente por encima del nivel del proyecto.

La Figura 2-1 muestra la organización del proyecto en relación con las responsabilidades SQA.

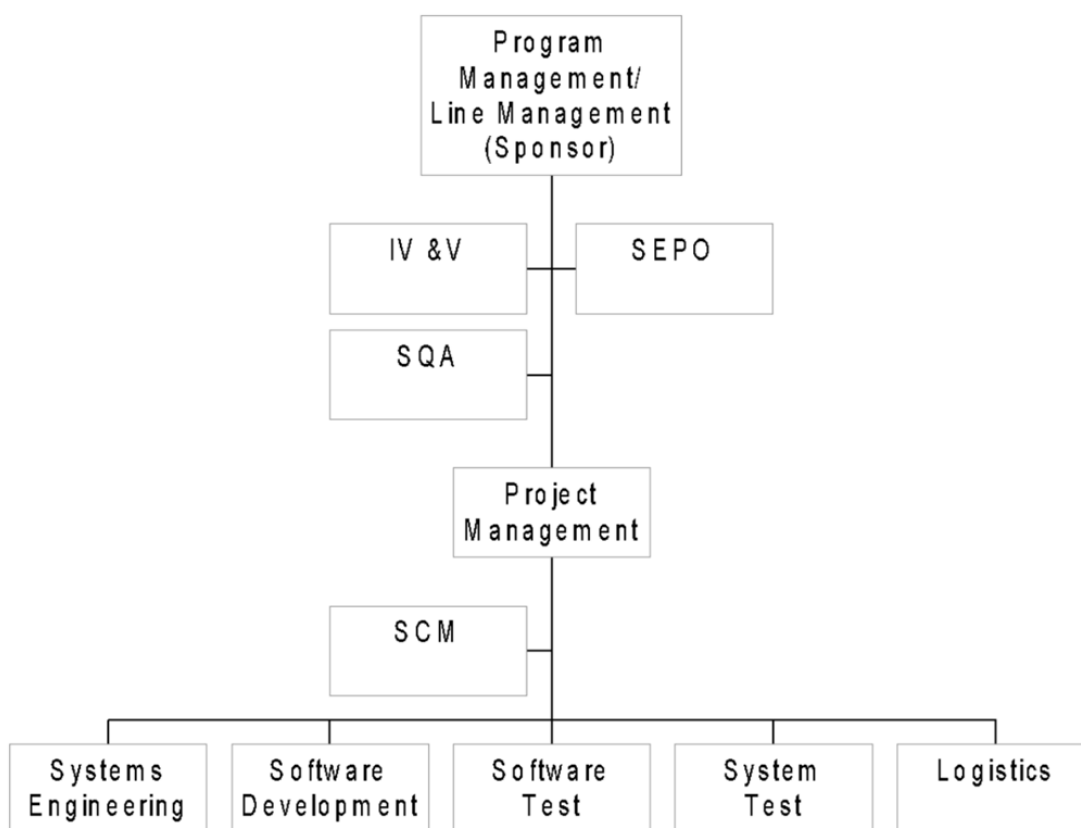


Figura 2: Organización del Proyecto Galería de Imágenes Segura

SQA es responsable de asegurar el cumplimiento de los requisitos de SQA delineados en este Plan SQA. La organización SQA asegura la calidad del software entregable y su documentación, el software no entregable, y los procesos de ingeniería utilizados para producir software.

A continuación se describen los grupos funcionales que influyen y controlan la calidad del software. La Tabla 3-2 del Proyecto define la matriz de responsabilidades asociada a

estos grupos.

1. Gerente SQA — es responsable de:

- a) Implementar y mantener el Plan SQA del proyecto a lo largo de todo el ciclo de vida del desarrollo.
- b) Auditar los procesos de desarrollo de software para verificar el cumplimiento de los estándares, procedimientos y este Plan SQA.
- c) Revisar y evaluar los productos de software (entregables y no entregables) para asegurar la calidad.
- d) Registrar y reportar los resultados de las evaluaciones y auditorías de SQA a la Gestión del Proyecto.
- e) Coordinar las revisiones por pares de los productos de software según el Proceso de Revisión por Pares, referencia (n).
- f) Dar seguimiento a los hallazgos de calidad y verificar que las acciones correctivas se implementen y cierren.
- g) Revisar las mediciones de SQA (costo, cronograma, elementos de incumplimiento) y reportarlas mensualmente.

2. Gerente de Proyecto — [Nombre] es responsable de:

- a) Implementar el programa de calidad de acuerdo con los estándares del proyecto.
- b) Identificar las actividades SQA a ser realizadas por el Gerente SQA.
- c) Revisar y aprobar el Plan SQA del proyecto.
- d) Resolver y dar seguimiento a cualquier problema de calidad planteado por SQA.
- e) Identificar y asegurar los factores de calidad a implementar en el sistema y el software.
- f) Identificar, desarrollar y mantener documentos de planificación como el Plan de Desarrollo de Software, el Plan de Pruebas y este Plan SQA.
- g) Gestionar el cronograma, los recursos y la asignación de tareas del equipo.

3. Ingeniería de Sistemas — es responsable de:

- a) Revisar y comentar el Plan SQA del proyecto.
- b) Implementar el programa de calidad de acuerdo con este Plan SQA.
- c) Resolver y dar seguimiento a cualquier problema de calidad planteado por SQA relacionado con las actividades de ingeniería.
- d) Diseñar la arquitectura del sistema (backend Spring Boot, frontend React, persistencia PostgreSQL).
- e) Identificar e implementar los factores de calidad del sistema (seguridad, rendimiento, integridad).

f) Definir las interfaces entre componentes y verificar su correcta integración.

4. Diseño/Desarrollo de Software – es responsable de:

- a)* Revisar y comentar el Plan SQA del proyecto.
- b)* Implementar el programa de calidad de acuerdo con este Plan SQA.
- c)* Resolver y dar seguimiento a cualquier problema de calidad planteado por SQA relacionado con el diseño y desarrollo del software.
- d)* Implementar las prácticas, procesos y procedimientos de codificación definidos en la Sección 5.
- e)* Realizar pruebas unitarias del código implementado y registrar resultados.
- f)* Participar en revisiones de código y corregir defectos identificados.

5. Pruebas — es responsable de:

- a)* Revisar y comentar el Plan SQA del proyecto.
- b)* Implementar el programa de calidad de acuerdo con este Plan SQA.
- c)* Resolver y dar seguimiento a cualquier problema de calidad planteado por SQA relacionado con las pruebas.
- d)* Ejecutar los casos de prueba definidos en el Plan de Pruebas (CP01–CP38) y registrar resultados.
- e)* Verificar que los factores de calidad se implementen en el sistema, específicamente en el software.
- f)* Generar informes de cobertura (JaCoCo), análisis estático (SonarQube, Qlty) y pruebas de seguridad (OWASP ZAP).

2.2 RECURSOS

2.2.1. Instalaciones y Equipos

Cuadro 4: Recursos del Proyecto

RECURSO	DESCRIPCIÓN	USO
GitHub	Repositorio remoto del proyecto	Control de versiones (SCM), gestión de issues, revisiones de código
Docker & Docker Compose	Entorno de contenedores	SonarQube (análisis estático), ClamAV (escaneo de malware), despliegue del backend
Vercel	Plataforma de despliegue	Hospedaje del frontend React (producción)
Java 21 + Maven (mvnw)	Runtime y herramienta de compilación	Desarrollo y compilación del backend Spring Boot
Node.js 20+ & npm	Runtime y gestor de paquetes	Desarrollo del frontend React, ejecución de pruebas Cypress
PostgreSQL 14+	Sistema de gestión de bases de datos	Persistencia del sistema (producción)
H2 Database	Base de datos en memoria	Ejecución de pruebas unitarias e integración (perfil test)
Entorno de desarrollo local	IDE, editor de código	Codificación, depuración, revisiones por pares

2.2.2. Personal

Cuadro 5: Estructura del Equipo del Proyecto

ROL	NOMBRE	CÉDULA	PORCENTAJE DE ESFUERZO
Gerente SQA	Gabriel López	[Cédula]	100 %
Ingeniería de Sistemas	Mateo Llumigusín	[Cédula]	100 %
Pruebas	Fernando Tipán	[Cédula]	100 %
Pruebas	Kevin Asmal	[Cédula]	100 %
Pruebas	Marcelo Pareja	[Cédula]	100 %

2.2.3. Calificaciones del Personal

El Gerente de SQA deberá estar familiarizado y poder aplicar los estándares y pautas enumerados en la Sección 1.6. Además, el Gerente de SQA deberá estar familiarizado con la calidad del software, las actividades relacionadas con el desarrollo de software y el análisis estructurado, diseño, codificación y pruebas.

El personal de Desarrollo de Software deberá tener experiencia en Java/Spring Boot (backend) y JavaScript/React (frontend), así como conocimiento de los estándares de codificación definidos en la Sección 5.

El personal de Pruebas deberá estar familiarizado con las herramientas de prueba definidas en la Sección 8 (JUnit, Cypress, JaCoCo, SonarQube, OWASP ZAP, JMeter, Lighthouse) y con la ejecución y documentación de casos de prueba.

SECCIÓN 3. TAREAS DE SQA

El SQA se encargará de evaluar la calidad del sistema tanto en la capa de presentación como en los endpoints del sistema, con mediciones de rendimiento, eficiencia, seguridad, efectividad y corrección.

3.1 TAREA: REVISIÓN DE PRODUCTOS DE SOFTWARE

Dentro del sistema evaluado, se encontró poca documentación para analizar; apenas se definen las funciones principales, las tecnologías utilizadas y la configuración inicial del sistema. No tiene pruebas definidas para comprobar la total funcionalidad del sistema, por lo que la mayoría de las pruebas y evaluaciones se realizarán a partir de las pruebas adaptadas al software entregado a través del plan generado para dichas pruebas.

3.2 TAREA: REVISIÓN DE HERRAMIENTAS DE SOFTWARE

Se definirán las herramientas utilizadas para las mediciones, tanto en qué parte del proceso se utilizarán como para qué tipos de prueba se usarán. Todas estas pruebas serán nuevas en el proyecto al no tener definido inicialmente antes del análisis. Cada una está pensada para el testeo en el ambiente en el que el sistema fue desarrollado, como son Java y TypeScript, que definen el servicio web.

3.3 TAREA: EVALUAR PROCESO DE REVISIÓN DE PRODUCTOS DE SOFTWARE

Esta tarea de SQA asegura que los procesos de revisión de calidad estén establecidos para todos los productos de software, que pueden incluir representaciones de información distintas a los documentos tradicionales en papel, y que estos productos hayan pasado por evaluación, pruebas y acciones correctivas según lo requiera el estándar.

SQA verificará que los productos de software listos para revisión sean revisados, que los resultados se reporten y que los problemas reportados se resuelvan de acuerdo con la referencia (e).

Los resultados de esta tarea se documentarán utilizando el Formulario de Auditoría de Procesos descrito en la Sección 7 y se proporcionarán a la gestión del proyecto. La recomendación de SQA para la acción correctiva requiere la disposición de la gestión del proyecto y se procesará de acuerdo con la guía en la Sección 7.

3.4 TAREA: EVALUAR PROCESOS DE PLANIFICACIÓN, MONITOREO Y SUPERVISIÓN DE PROYECTOS

La planificación, seguimiento y supervisión del proyecto implica que la gestión del proyecto desarrolle y documente planes para el Desarrollo de Software, Pruebas de CI y Sistemas, Instalación de Software y Transición de Software. La Sección 1.6 enumera los planes del programa/proyecto. Para los documentos del proyecto que se desarrollarán, SQA ayudará a identificar las pautas, estándares o Descripciones de Elementos de Datos (DIDs) apropiados, y ayudará con la adaptación de esas pautas, estándares o DIDs para satisfacer las necesidades del proyecto.

SQA evaluará que el proyecto realice las actividades relevantes establecidas en los planes del Programa y del Proyecto. Para verificar que estas actividades se realicen según lo planificado, SQA auditará los procesos que definen la actividad y utilizará la referencia (e) u otro documento de planificación como medida de cumplimiento.

SQA utilizará las listas de verificación de auditoría en las Figuras B-1 y B-2 como guías para realizar la evaluación.

Los resultados de esta tarea se documentarán utilizando el Formulario de Auditoría de Procesos descrito en la Sección 7. Cualquier cambio recomendado a esos planes requerirá actualización y aprobación por parte de la gestión del proyecto de acuerdo con el procedimiento de gestión de configuración descrito en el Plan SCM del proyecto.

3.5 TAREA: EVALUAR PROCESOS DE IMPLEMENTACIÓN, PRUEBAS UNITARIAS E INTEGRACIÓN

La implementación o codificación del software es el punto en el ciclo de vida del desarrollo en el que el diseño finalmente se implementa. El proceso incluye pruebas unitarias del código de software, integración de componentes y pruebas de calificación de CI.

En la fase de integración y prueba del ciclo de vida del desarrollo, el enfoque de las pruebas cambia de la corrección de un componente individual al funcionamiento adecuado de las interfaces entre componentes, el flujo de información a través del sistema y el cumplimiento de los requisitos del sistema.

Las actividades de SQA se enumeran a continuación:

1. Verificar que el proceso de codificación, las revisiones de código asociadas y las pruebas unitarias de software se realicen de acuerdo con los estándares y procedimientos establecidos por el proyecto y descritos en la Sección 5.
2. Verificar que los elementos de acción resultantes de las revisiones del código se resuelvan de acuerdo con estos estándares y procedimientos.
3. Verificar que las actividades de prueba de software estén identificadas, que los entornos de prueba estén definidos y que se hayan diseñado pautas para las pruebas.
4. Verificar que el proceso de integración de software, las actividades de prueba de integración de software y las actividades de prueba de rendimiento del software se realicen de acuerdo con el plan de pruebas y los estándares del proyecto.
5. Verificar que se presencien las pruebas de integración de software y las pruebas de rendimiento del software para verificar que se sigan los procedimientos de prueba aprobados, que se mantengan registros precisos de los resultados de las pruebas, que todas las discrepancias descubiertas durante las pruebas se informen correctamente y que se completen los informes de prueba asociados.
6. Verificar que las discrepancias descubiertas durante las pruebas de integración y rendimiento del software se identifiquen, analicen, documenten y corrijan; que las pruebas unitarias y de integración del software se vuelvan a ejecutar según sea necesario para validar las correcciones realizadas al código.

7. Verificar que la responsabilidad de las pruebas y de la presentación de informes sobre los resultados se haya asignado a un elemento organizativo específico.
8. Revisar el Plan de Pruebas de Software y las Descripciones de Pruebas de Software para verificar el cumplimiento de los requisitos y estándares.
9. Monitorear las actividades de prueba, presenciar pruebas y certificar los resultados de las pruebas.
10. Verificar que se hayan establecido requisitos para la certificación o calibración de todo el software o hardware de soporte utilizado durante las pruebas.

SQA puede utilizar las listas de verificación de auditoría en las Figuras B-7, B-8 y B-9 como guías para realizar estas evaluaciones.

Los resultados de esta tarea se documentarán utilizando el Formulario de Auditoría de Procesos descrito en la Sección 7 y se proporcionarán a la gestión del proyecto. La recomendación de SQA para la acción correctiva requiere la disposición de la gestión del proyecto y se procesará de acuerdo con la guía en la Sección 7.

3.6 TAREA: EVALUAR PROCESOS DE CONTROL DE CÓDIGO

El control de código incluye los métodos e instalaciones utilizados para mantener, almacenar, asegurar y documentar las versiones controladas del software identificado durante todas las fases del ciclo de vida del software. El proyecto utiliza GitHub como repositorio remoto para el control de versiones del código fuente.

Las actividades de SQA se enumeran a continuación:

1. Verificar que el repositorio de código esté establecido y que existan procedimientos para gobernar su operación (ramas, pull requests, merge).
2. Verificar que la documentación y los materiales del programa informático estén aprobados y colocados bajo control de gestión de configuración en el repositorio.
3. Verificar el establecimiento de procedimientos de liberación formal para la documentación y versiones de software aprobadas.
4. Verificar que los controles del repositorio prevengan cambios no autorizados en el software controlado (protección de ramas, revisiones obligatorias) y verificar la incorporación de todos los cambios aprobados.

SQA puede utilizar la lista de verificación de auditoría en la Figura B-18 como guía para realizar esta evaluación.

Los resultados de esta tarea se documentarán utilizando el Formulario de Auditoría de Procesos descrito en la Sección 7 y se proporcionarán a la gestión del proyecto. La recomendación de SQA para la acción correctiva requiere la disposición de la gestión del proyecto y se procesará de acuerdo con la guía en la Sección 7.

Los informes de SQA, junto con los registros de acción correctiva, los informes de pruebas de software y los registros de evaluación de productos de software, constituyen la evidencia requerida para certificar que el software desempeña las funciones previstas.

3.7 TAREA: VERIFICAR REVISIONES DE PROCESOS Y AUDITORÍAS

El tipo y alcance de las revisiones técnicas depende en gran medida del tamaño, alcance, riesgo y criticidad del proyecto de software. Para este proyecto académico, se definen las siguientes revisiones y auditorías adaptadas al alcance del sistema.

3.7.1. Tarea: Verificar Revisiones Técnicas

Un componente principal de la ingeniería de calidad en el software es la realización de revisiones técnicas de los productos de software, tanto entregables como no entregables. Los participantes de una revisión técnica deben incluir personas con conocimiento técnico de los productos de software a revisar. El propósito de la revisión técnica será centrarse en los productos de software en progreso y finales, en lugar de los materiales generados especialmente para la revisión.

Para este proyecto se definen las siguientes revisiones técnicas:

1. Revisión por Pares: el objetivo es revisar el código fuente, la documentación y los productos de software en progreso para identificar defectos, verificar el cumplimiento de estándares y obtener retroalimentación técnica. Se realizará antes de fusionar cambios en la rama principal.
2. Revisión de Diseño: el objetivo es evaluar la arquitectura del sistema (backend Spring Boot, frontend React, persistencia PostgreSQL), las interfaces entre componentes y la adecuación de la solución técnica a los requisitos de seguridad y funcionalidad.

Las revisiones técnicas se llevarán a cabo para revisar los productos de software en evolución, demostrar las soluciones técnicas propuestas y proporcionar información y obtener retroalimentación sobre el esfuerzo técnico. El resultado de una revisión técnica se enumera a continuación:

1. Identificar y resolver problemas técnicos.
2. Revisar el estado del proyecto, específicamente los riesgos a corto y largo plazo en cuanto a aspectos técnicos, costos y plazos.
3. Llegar a estrategias de mitigación acordadas para los riesgos identificados, dentro de la autoridad de los presentes.
4. Identificar riesgos y problemas que se plantearán en las revisiones conjuntas de gestión.
5. Verificar la comunicación continua entre los miembros del equipo del proyecto.

Los criterios de entrada para una revisión técnica requerirán que un elemento a revisar se distribuya al grupo antes de la reunión de revisión. Además, se asignará un secretario para registrar cualquier problema que requiera resolución, indicando el responsable del elemento de acción y la fecha de vencimiento, y las decisiones tomadas dentro de la autoridad de los participantes de la revisión técnica.

Se recopilan varias mediciones como parte de las revisiones técnicas para ayudar a determinar la efectividad del proceso de revisión en sí, así como de los pasos del proceso que se utilizan para producir el elemento que se revisa. Estas mediciones, reportadas al gerente del proyecto, incluirán la cantidad de tiempo dedicado por cada persona involucrada en la revisión, incluida la preparación para la revisión.

3.7.2. Tarea: Verificar Revisiones de Gestión

La revisión periódica de la gestión de SQA del estado, progreso, problemas y riesgos del proyecto de software proporcionará una evaluación independiente de las actividades del proyecto. SQA proporcionará la siguiente información a la gestión:

1. **Cumplimiento:** Identificación del nivel de cumplimiento del proyecto con los procesos organizacionales y del proyecto establecidos.
2. **Áreas problemáticas:** identificación de áreas problemáticas potenciales o reales del proyecto basadas en el análisis de los resultados de las revisiones técnicas.
3. **Riesgos:** identificación de riesgos basados en la participación y evaluación del progreso del proyecto y las áreas problemáticas.

Debido a que la función SQA es integral para el éxito del proyecto, SQA comunicará libremente sus resultados a la dirección del proyecto y al equipo del proyecto. El método para informar el cumplimiento, las áreas problemáticas y los riesgos se comunicará en un informe documentado o memorando. Se dará seguimiento a los problemas de cumplimiento, áreas problemáticas y riesgos y se rastrearán hasta su cierre.

3.7.3. Tarea: Conducir Auditorías de Procesos

Los procesos de desarrollo de software se auditan de acuerdo con las tareas especificadas en esta Sección y se realizan de acuerdo con el cronograma de desarrollo de software. SQA utilizará las listas de verificación de auditoría del Apéndice B como guías para realizar las evaluaciones.

Los resultados de esta tarea se documentarán utilizando el Formulario de Auditoría de Procesos descrito en la Sección 7 y se proporcionarán a la gestión del proyecto.

3.7.4. Tarea: Realizar Auditorías de Configuración

Auditoría de Configuración Funcional (FCA). La Auditoría de Configuración Funcional se lleva a cabo antes de la entrega del software para comparar el software construido con los requisitos de software establecidos. El propósito es asegurar que el código aborde todos, y solo, los requisitos documentados y las capacidades funcionales establecidas. SQA participará como miembro del equipo de FCA. SQA ayudará en la preparación de los hallazgos de la FCA. Cualquier seguimiento de los hallazgos reportados de la FCA será monitoreado y rastreado hasta su cierre.

Auditoría de Configuración Física (PCA). La Auditoría de Configuración Física se lleva a cabo para verificar que el software y su documentación son internamente consistentes

y están listos para la entrega. El propósito es asegurar que la documentación que se va a entregar sea consistente y describa correctamente el código. SQA participará como miembro del equipo de PCA. SQA ayudará en la preparación de los hallazgos de la PCA. Cualquier seguimiento de los hallazgos reportados de la PCA será monitoreado y rastreado hasta su cierre.

3.8 RESPONSABILIDADES

La responsabilidad última de la calidad del software y la documentación asociada del proyecto recae en el Gerente de Proyecto de Software del proyecto. El Gerente de SQA implementará los procedimientos de SQA definidos en este plan.

SQA deriva su autoridad del Gerente del Proyecto. SQA monitoreará las actividades del personal del proyecto y revisará los productos para verificar el cumplimiento de los estándares y procedimientos aplicables. Los resultados del monitoreo y análisis de SQA, junto con las recomendaciones de SQA para acciones correctivas, se reportarán al Gerente de Proyecto de Software. Todos los documentos y software aprobados por el Gerente de Proyecto de Software para su liberación deberán haber sido revisados y aprobados por SQA. La Tabla 3-1 es una matriz de responsabilidades para las tareas identificadas en esta Sección.

Cuadro 6: Matriz de Responsabilidades por Proceso

Proceso	Ger. SQA	Ger. Prog.	Ger. Proy.	Ing. Sist.	Des. SW	Probador
Desarrollar/Documentar Plan SQA	X		X			
Revisar Plan SQA	X	X	X	X	X	X
Aprobar Plan SQA	X	X	X			
Revisar productos de software	X	X	X	X	X	X
Aprobar producto		X	X			
Evaluar herramientas de software	X				X	X
Desarrollar/Documentar planes del proyecto		X	X			
Revisar planes del proyecto	X	X	X	X	X	X
Evaluar proceso de planificación	X					
Desarrollar/Corregir código					X	
Revisar código	X				X	X
Prueba unitaria					X	X
Integrar software					X	
Probar software integrado						X
Prueba de rendimiento						X
Evaluar/Reportar proceso de implementación y pruebas	X					
Control de código (GitHub)	X				X	
Revisar requisitos del sistema	X	X	X	X	X	X
Revisar requisitos de software	X	X	X	X	X	X
Revisar diseño de software	X	X	X	X	X	X
Auditorías de procesos	X					
Auditorías de configuración	X			X	X	X
Seguir proceso de acción correctiva	X	X	X	X	X	X
Resolver hallazgos de auditoría		X	X			
Asistir a auditorías SQA	X			X	X	X
Nombrar auditor SQA independiente		X				

3.9 CRONOGRAMA

Los cronogramas de SQA están estrechamente coordinados con el cronograma de desarrollo de software. El proyecto se ejecutó en un período de dos semanas con las siguientes fases:

Cuadro 7: Cronograma del Proyecto

FASE	ACTIVIDADES SQA	PRODUCTOS
Semana 1: Revisión y Análisis	Revisión del sistema existente, identificación de herramientas SQA, definición del plan de pruebas, auditoría de procesos iniciales	SQAP, Plan de Pruebas (STP), definición de CP01–CP38, matriz de trazabilidad
Semana 2: Pruebas y Documentación	Ejecución de casos de prueba (CP01–CP38), generación de informes (JaCoCo, SonarQube, ZAP, Lighthouse, Cypress), registro de hallazgos y acciones correctivas, elaboración de documentación SQA	Informe de Pruebas (STR), cobertura JaCoCo, hallazgos SEC-01/XSS-01/CFG-01/BLD-01/CAL-01, RESULTADOS_CP30_CP39

Las auditorías de procesos se realizarán al comienzo de cada nueva fase de desarrollo para verificar que los procesos apropiados estén correctamente implementados según lo definido en los documentos de planificación. Además, se realizarán verificaciones puntuales (auditorías no programadas) durante cada fase de desarrollo para verificar que se sigan los procesos y los procedimientos de escritorio. Al finalizar una fase de desarrollo de software, SQA revisará e informará si se han completado todos los pasos necesarios para la transición a la siguiente fase.

SECCIÓN 4. DOCUMENTACIÓN

La documentación que describe y respalda el software del proyecto o el proceso de desarrollo de software se creará y actualizará periódicamente durante todo el ciclo de desarrollo.

La Tabla 4-1 es una lista de los productos de software entregables del proyecto y el estándar o pautas asociados utilizados para desarrollar y mantener los productos de software. La Tabla 4-2 es una lista de productos no entregables.

Para los documentos de software del proyecto que se desarrollarán y que aún no figuran en las Tablas 4-1 y 4-2, SQA ayudará a identificar las especificaciones, estándares y Descripciones de Elementos de Datos (DIDs) que se seguirán en la preparación de la documentación requerida.

Cuadro 8: Productos de Software Entregables

NOMENCLATURA	DOCUMENTACIÓN ENTREGABLE	DID, ESTÁNDAR, PAUTA
CI-01	Plan de Aseguramiento de Calidad de Software (SQAP) — secciones 1–14, apéndices A y B	IEEE Std 730-1998
CI-04	Plan de Pruebas de Software (STP) — CP01–CP38, matriz de trazabilidad, criterios de aceptación	DI-IPSC-81435 (MIL-STD-498) adaptado
CI-05	Informe de Pruebas de Software (STR) — ejecución de CP, cobertura JaCoCo, hallazgos y análisis	DI-IPSC-81436 (MIL-STD-498) adaptado
CI-06	Documentación de Usuario — README.md, guía de instalación y configuración	IEEE Std 1063-2001
CI-07	Guía de Despliegue — Dockerfile, docker-compose, instrucciones de producción	Interna, alineada a IEEE 12207

Cuadro 9: Productos de Software No Entregables

TÍTULO DEL DOCUMENTO
Código fuente backend (Java/Spring Boot) — src/main/java/
Código fuente frontend (React/Vite) — frontend/src/
Configuración de la aplicación — application.yaml, application-test.yaml
Scripts de compilación y ejecución — mvnw, mvnw.cmd, pom.xml
Scripts de prueba — comandos pruebas.txt, detect_steganography.cmd
Configuración de análisis estático — sonar-project.properties, docker-compose.sonar.yml
Configuración de seguridad dinámica — zap-api-scan.bat, zap-api-scan.sh
Configuración de despliegue — vercel.json, lighthousec.js
Plan de pruebas de carga — jmeter/quarantine-load-test.jmx
Formularios de auditoría — Apéndice B (listas de verificación)
Formulario de Reporte de Discrepancias/Cambios — DCR Form

1. Todos los documentos se someterán a una revisión por pares de acuerdo con el Proceso de Revisión por Pares, referencia (n).
2. El SQAP (CI-01) será revisado por todo el equipo y aprobado por el Gerente de Proyecto antes de la fase de planificación.
3. El Plan de Pruebas (CI-04) será revisado antes de la ejecución de pruebas para verificar que los criterios de aceptación y la trazabilidad estén correctamente definidos.
4. El Informe de Pruebas (CI-05) será revisado tras la ejecución de pruebas, registrando hallazgos en el Formulario de Reporte de Discrepancias (DCR) del Apéndice.
5. La documentación de usuario (CI-06) y de despliegue (CI-07) serán verificadas mediante revisión por pares antes de su liberación.
6. Los productos no entregables (código fuente, configuraciones, scripts) serán verificados continuamente mediante revisiones de código, análisis estático (SonarQube, Qlty) y herramientas de cobertura (JaCoCo).

Tras la finalización de una revisión por pares, SQA registra y reporta las mediciones de la revisión por pares (el elemento revisado, el número de errores detectados, la fase en la que se realizó la revisión por pares, el número de informes de error cerrados y el número de informes de error abiertos) a SEPO.

Tras la finalización de una revisión por pares, el producto de software se enviará a SCM y se colocará bajo control de CM. El producto de software se procesará entonces de acuerdo con el proceso de aprobación y liberación de productos de software de SCM descrito en la referencia (e).

SECCIÓN 5. ESTÁNDARES, PRÁCTICAS, CONVENCIONES Y MÉTRICAS

Para verificar la entrega de un producto totalmente conforme y de alta calidad, cada individuo asignado al proyecto participará en el aseguramiento de la calidad. La referencia (e) define los procedimientos mediante los cuales el personal de desarrollo de software verificará la calidad del producto durante el proceso de desarrollo. El resto de esta sección describe los procedimientos utilizados por SQA para verificar que se cumplan las disposiciones de aseguramiento de la calidad de este Plan SQA y los estándares, prácticas, convenciones y métricas aplicables.

El presente Plan SQA define los estándares, prácticas, convenciones y métricas aplicables al proyecto Galería de Imágenes Segura. La Sección 3 identifica la evaluación de SQA de las fases de implementación y prueba para verificar el cumplimiento de estos estándares.

La Sección 4 identifica los Configuration Items (CIs) asociados a cada producto de software que se desarrollará. SQA verificará que el formato y el contenido de la documentación cumplan con los criterios definidos en el SQAP.

Los estándares para la estructura lógica, la codificación y los comentarios del código se describen en las convenciones del proyecto (Google Java Style Guide para backend, Airbnb Style Guide para frontend). SQA verificará que el código fuente cumpla con estos estándares.

Los estándares y prácticas para las pruebas se describen en el Plan de Pruebas de Software (STP, CI-04). SQA verificará que las actividades de prueba cumplan con lo establecido.

5.1 MÉTRICAS

Identificar o hacer referencia a los estándares, prácticas y convenciones que se utilizarán en la definición, recopilación y utilización de datos de medición de software. Citar cualquier requisito o estándar interno (ej., del proyecto, corporativo) y externo (ej., del usuario, cliente) con el que deban cumplir las prácticas de métricas. IEEE Std 1045-1992, IEEE Standard for Software Productivity Metrics, referencia (p), describe convenciones para contar los resultados de los procesos de desarrollo. IEEE Std 1061-1992, IEEE Standard for a Software Quality Metrics Methodology, referencia (q), proporciona una metodología para seleccionar e implementar métricas de proceso y producto. IEEE Std 982.1-1988, IEEE Standard Dictionary of Measures to Produce Reliable Software, referencia (r), e IEEE Std 982.2-1988, IEEE Guide for using reference (r), referencia (s), proporcionan varias medidas para su uso en diferentes fases del ciclo de vida para ganar confianza en la construcción de software confiable. Para mantener las métricas simples, se ofrece un ejemplo de métricas de costo y cronograma.

Las siguientes mediciones se realizarán y utilizarán para determinar el estado de costo y cronograma de las actividades de SQA:

1. Fechas de hitos de SQA (planificadas)
2. Fechas de hitos de SQA (completadas)
3. Trabajo de SQA programado (planificado)

4. Trabajo de SQA completado (real)
5. Esfuerzo de SQA dedicado (planificado)
6. Esfuerzo de SQA dedicado (real)
7. Fondos de SQA gastados (planificados)
8. Fondos de SQA gastados (reales)
9. Número de elementos de incumplimiento abiertos
10. Número de elementos de incumplimiento cerrados
11. Número total de elementos de incumplimiento

SQA es responsable de reportar estas mediciones al Gerente de Proyecto mensualmente.

5.2 MÉTRICAS DE CALIDAD EN USO

La norma ISO/IEC 25022 define métricas para medir la calidad en uso de un producto de software desde la perspectiva del usuario final. Las métricas presentadas a continuación se organizan según las cinco características definidas en dicha norma: efectividad, eficiencia, satisfacción, libertad de riesgo y cobertura de contexto.

5.2.1. Efectividad

Precisión e integridad con las que los usuarios alcanzan objetivos específicos.

1. Tasa de finalización de tareas (Task completion rate)

$$TCR = \frac{N_{tareas_completadas}}{N_{tareas_intentadas}} \times 100\%$$

Donde $N_{tareas_completadas}$ es el número de tareas (pruebas, auditorías) completadas exitosamente y $N_{tareas_intentadas}$ es el total de tareas ejecutadas.

2. Frecuencia de errores (Error frequency)

$$FE = \frac{N_{errores}}{N_{ejecuciones}}$$

Donde $N_{errores}$ es el número de ejecuciones que presentaron errores y $N_{ejecuciones}$ es el número total de ejecuciones realizadas.

3. Tasa de logros de efectividad (Effectiveness achievement rate)

$$EAR = \frac{N_{audits_aprobados}}{N_{audits_totales}}$$

Donde $N_{audits_aprobados}$ son las auditorías que cumplen el umbral definido y $N_{audits_totales}$ es el total de auditorías ejecutadas.

4. Calidad percibida del producto

$$CP = \frac{\sum_{i=1}^n w_i \cdot s_i}{\sum_{i=1}^n w_i}$$

Donde w_i es el peso asignado a cada categoría de calidad y s_i es el puntaje obtenido en esa categoría (escala 0–1).

5.2.2. Eficiencia

Recursos gastados en relación con la precisión e integridad con las que los usuarios alcanzan los objetivos.

1. Tiempo de tarea (Task time)

$$TT = t_{fin} - t_{inicio}$$

Medición directa del tiempo transcurrido desde el inicio hasta la finalización de una tarea, expresado en milisegundos.

2. Índice de eficiencia de carga (Load efficiency index)

$$LEI = \frac{LCP}{TTI}$$

Donde LCP es el Largest Contentful Paint y TTI es el Time to Interactive. Un valor cercano a 1 indica carga continua sin bloqueos.

3. Tiempo de respuesta del sistema

$$TR = RTT + L_{servidor}$$

Donde RTT es el Round Trip Time de red y $L_{servidor}$ es la latencia del servidor backend.

4. Peso de recurso por elemento DOM

$$PR = \frac{B_{totales}}{N_{DOM}}$$

Donde $B_{totales}$ es el peso total en bytes de los recursos cargados y N_{DOM} es el número de elementos en el DOM.

5. Proporción de tiempo de ejecución JavaScript

$$PJS = \frac{T_{bootup}}{T_{mainthread}} \times 100 \%$$

Donde T_{bootup} es el tiempo de ejecución de JavaScript y $T_{mainthread}$ es el tiempo total de trabajo del hilo principal.

5.2.3. Satisfacción

Grado en que las necesidades del usuario se satisfacen cuando se utiliza el producto.

1. Puntaje de satisfacción del usuario (User satisfaction score)

$$USS = \frac{\sum_{i=1}^n R_i}{n}$$

Donde R_i son las calificaciones individuales obtenidas mediante encuestas (escala Likert 1–5) y n es el número de encuestados.

2. Net Promoter Score (NPS)

$$NPS = P_{promotores} - P_{detractores}$$

Donde $P_{promotores}$ es el porcentaje de usuarios que califican 9–10 y $P_{detractores}$ el porcentaje que califica 0–6.

3. Tasa de utilidad percibida (Perceived usefulness)

$$UP = \frac{N_{funcionalidades_utilizadas}}{N_{funcionalidades_disponibles}} \times 100\%$$

Nota: Las métricas de satisfacción requieren la aplicación de instrumentos de recolección como encuestas SUS (System Usability Scale) o cuestionarios personalizados. Se planifica su implementación en fases posteriores.

5.2.4. Libertad de Riesgo

Grado en que un producto mitiga los riesgos económicos, de salud, seguridad o medio ambiente.

1. Tasa de mitigación de riesgos de seguridad (Security risk mitigation)

$$SRM = 1 - \frac{V_{criticas} \cdot 10 + V_{altas} \cdot 5 + V_{medias} \cdot 2}{N_{endpoints}}$$

Donde $V_{criticas}$, V_{altas} y V_{medias} son vulnerabilidades encontradas según su severidad y $N_{endpoints}$ es el número de endpoints auditados.

2. Índice de riesgo frontend

$$IRF = 1 - BP$$

Donde BP es el puntaje de Best Practices obtenido en Lighthouse (escala 0–1).

3. Vulnerabilidad operativa detectada

$$VO = \frac{N_{errores_consola}}{N_{paginas_auditadas}}$$

Donde $N_{errores_consola}$ es el número de páginas que presentan errores en la consola del navegador.

4. Densidad de vulnerabilidades en dependencias

$$DVD = \frac{N_{vulnerabilidades_npm}}{N_{dependencias}}$$

Donde $N_{vulnerabilidades_npm}$ es el conteo de vulnerabilidades reportadas por npm audit y $N_{dependencias}$ es el total de dependencias del proyecto frontend.

5.2.5. Cobertura de Contexto

Grado en que un producto puede ser utilizado en contextos específicos o más allá de los inicialmente previstos.

1. Completitud de cobertura de pruebas

$$CCP = \frac{M_{cubiertos}}{M_{totales}} \times 100 \%$$

Donde $M_{cubiertos}$ son los módulos o capas del sistema que tienen pruebas automatizadas y $M_{totales}$ es el total de módulos identificados (unitarias, integración, E2E, carga, seguridad).

2. Madurez de aseguramiento de calidad

$$MA = \frac{C_{ejecutadas}}{C_{totales}} \times 100 \%$$

Donde $C_{ejecutadas}$ son las capas de prueba con resultados disponibles y $C_{totales}$ son las capas definidas en el plan de pruebas.

3. Índice de accesibilidad

$$IA = A$$

Donde A es el puntaje de accesibilidad obtenido en Lighthouse (escala 0–1), que refleja la capacidad del producto para ser utilizado por personas con discapacidades.

4. Tasa de adaptabilidad a dispositivos

$$TAD = \frac{N_{viewport_configurados}}{N_{pantallas_soportadas}}$$

Donde $N_{viewport_configurados}$ son los puntos de ruptura responsivos implementados y $N_{pantallas_soportadas}$ son las resoluciones objetivo definidas en los requisitos.

Métricas de Calidad de Producto

La norma ISO/IEC 25023 define métricas para medir la calidad de un producto o sistema de software de acuerdo con las características y sub-características de la ISO 25010. Las métricas presentadas a continuación se organizan según algunas de las características escogidas para medirse en este sistema: Eficiencia de desempeño, Seguridad y Mantenibilidad.

5.3.1. Eficiencia de rendimiento

Medidas centradas en el rendimiento y uso de recursos del sistema.

- **Tiempo medio por petición**

$$T_{med} = \frac{\sum t_{peticiones}}{N_{endpoints_probados}}$$

Donde $\sum t_{peticiones}$ es la suma de los tiempos registrados para las peticiones a los endpoints y $N_{endpoints_probados}$ es el número de endpoints evaluados.

- **Cantidad de recursos usados**

$$R_{avg} = \frac{\sum R_{peticiones}}{N_{endpoints_probados}}$$

Donde $\sum R_{peticiones}$ es la suma de los recursos consumidos (memoria, CPU u otras métricas normalizadas) por las peticiones y $N_{endpoints_probados}$ es el número de endpoints evaluados.

5.3.2. Seguridad

Métricas que cuantifican la integridad y confidencialidad del sistema.

- **Nivel de integridad del sistema**

$$I_{nivel} = 100 - \frac{N_{amenazas_integridad}}{N_{amenazas_totales}} \times 100$$

Donde $N_{amenazas_integridad}$ es el número de amenazas que afectan la integridad y $N_{amenazas_totales}$ es el total de amenazas identificadas.

- **Nivel de confidencialidad de datos**

$$C_{nivel} = 100 - \frac{N_{amenazas_confidencialidad}}{N_{amenazas_totales}} \times 100$$

Donde $N_{amenazas_confidencialidad}$ es el número de amenazas que afectan la confidencialidad.

5.3.3. Mantenibilidad

Indicadores relativos a la facilidad de mantenimiento y calidad del código.

- **Porcentaje de código duplicado**

$$CD = \frac{LOC_{duplicadas}}{LOC_{revisadas}} \times 100\%$$

- **Deuda técnica promedio por módulo**

$$DT_{avg} = \frac{\sum T_{deuda_tecnica}}{N_{modulos_revisados}}$$

Donde $\sum T_{deuda_tecnica}$ es la suma de las estimaciones (en horas o puntos) de la deuda técnica registrada por módulo.

- **Complejidad del código**

$$CC_{avg} = \frac{\sum CC_{ciclomatica}}{N_{modulos_revisados}}$$

Donde $\sum CC_{ciclomatica}$ es la suma de la complejidad ciclomática medida en los módulos revisados.

SECCIÓN 6. TESTS

Identificar todas las demás pruebas no incluidas en la verificación y validación y establecer los métodos utilizados. Describir cualquier técnica o método de prueba que se pueda utilizar para detectar errores, desarrollar conjuntos de datos de prueba y monitorear los recursos del sistema informático.

La actividad de prueba del proyecto incluye pruebas a nivel de unidad, pruebas de integración (a nivel de Unidad y CI/HWCI), pruebas de rendimiento (Pruebas de Calificación de CI) y pruebas de aceptación (Pruebas de Calificación del Sistema). La Figura 6-1 proporciona el Diagrama de Flujo del Proceso de Prueba. SQA auditará las actividades de prueba según lo definido en la referencia (e), y verificará que el software y la documentación de prueba estén sujetos al control de gestión de configuración. SQA presenciara las pruebas y verificará que los resultados de las pruebas se registren y evalúen. SQA coordinará el mantenimiento de los registros de Informes de Problemas/Cambios (P/CR), a veces llamados Informes de Problemas de Software (STR), con SCM y verificará que los cambios de software se controlen según la referencia (e). SQA presenciara las pruebas de regresión resultantes de P/CR o STR para verificar la efectividad de la corrección.

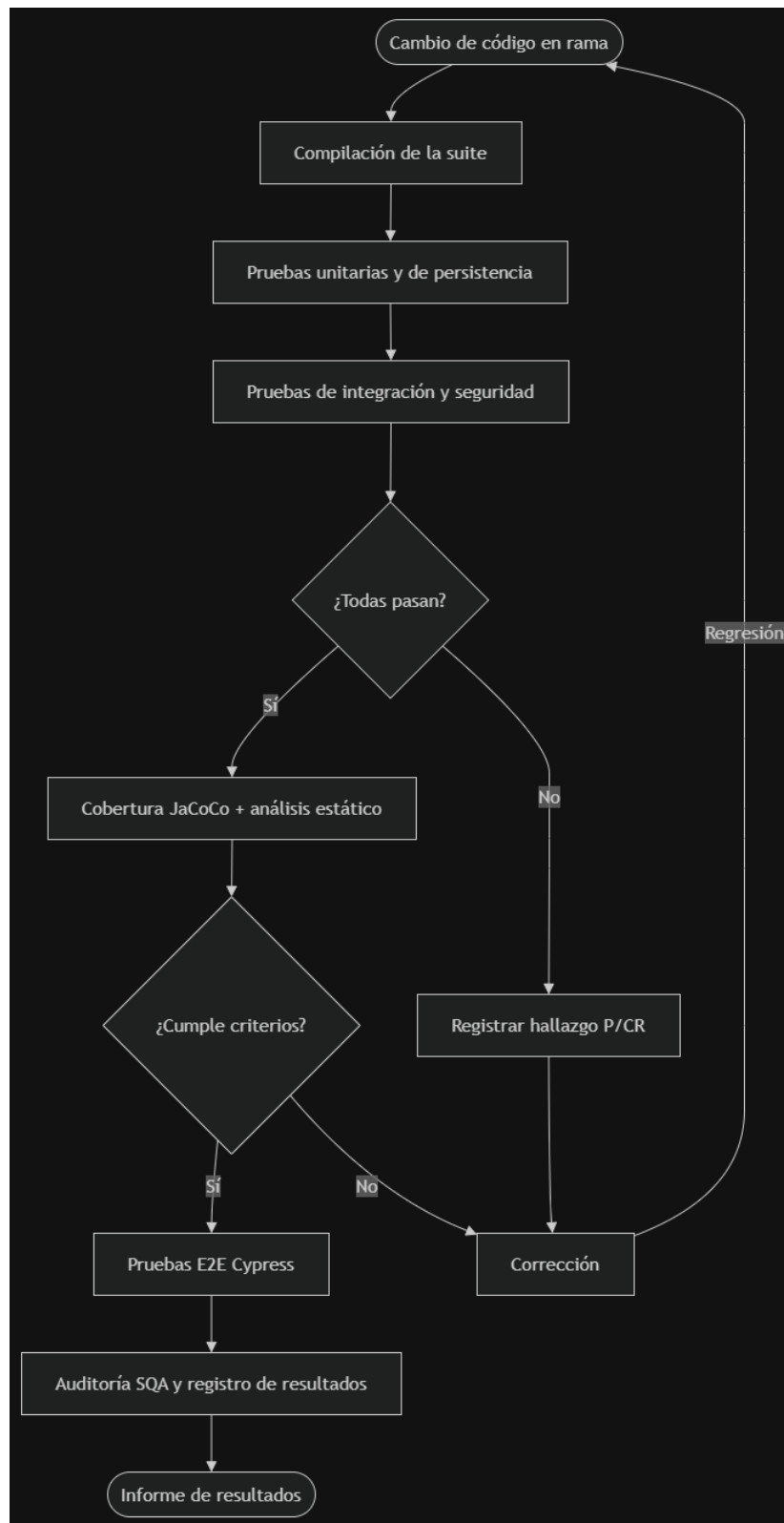


Figura 3: Diagrama de Flujo del Proceso de Prueba

6.1 SISTEMA BAJO PRUEBA Y NIVELES DE PRUEBA

El sistema bajo prueba (SUT) es una Galería de Imágenes Segura desarrollada con Spring Boot 4.0.6 sobre Java 21, frontend React 18 con Vite y persistencia en PostgreSQL. Su característica diferenciadora es la detección de contenido oculto mediante análisis de esteganografía (anomalías EOF, firmas embebidas y metadatos EXIF), con cuarentena de archivos sospechosos y un panel de supervisor para auditoría.

Las pruebas se organizan en los siguientes niveles:

Cuadro 10: Niveles de prueba y su implementación

NIVEL	ALCANCE	IMPLEMENTACIÓN
Unitario	Lógica de detección de esteganografía y validación de archivos	FileStorageServiceTest (JUnit 5)
Persistencia	Consultas y estados de las entidades Image y Album	ImageRepositoryTest, AlbumSanitizationJpaTest (@DataJpaTest sobre H2)
Integración	Endpoints REST con cadena de seguridad y control de acceso	SupervisorControllerTest, SupervisorPanelAuditTest, XssSanitizationTest, AlbumAuthorizationTest (MockMvc)
Seguridad	Sanitización XSS, autorización por rol, path traversal y CSRF	Distribuido entre las clases de integración
Sistema (E2E)	Flujos críticos del frontend	Cypress (quarantine-harmless, quarantine-harmful)
Estático	Deuda técnica, code smells y vulnerabilidades de código	SonarQube, Qlty, npm audit
Cobertura	Medición cuantitativa de la cobertura alcanzada	JaCoCo

6.2 MATRIZ DE TRAZABILIDAD

La siguiente matriz relaciona cada requisito funcional con los casos de prueba que lo verifican.

REQUISITO	CASO	DEFINICIÓN DEL CASO DE PRUEBA
RF-01 Registro y Autenticación Segura	CP01	Un usuario intenta registrarse usando datos válidos
	CP02	Un usuario intenta registrarse usando datos incorrectos o inválidos
	CP03	Un usuario con cuenta intenta iniciar sesión con credenciales válidas

REQUISITO	CASO	DEFINICIÓN DEL CASO DE PRUEBA
	CP04	Un usuario con cuenta intenta iniciar sesión con credenciales inválidas
	CP05	Un usuario intenta ingresar repetidamente con credenciales inválidas
RF-02 Control de Acceso y Sesión	CP06	Un usuario con rol USER intenta acceder a funcionalidades exclusivas del SUPERVISOR
	CP07	Un usuario con rol SUPERVISOR accede al panel de administración
	CP08	Un usuario no autenticado intenta acceder a un recurso protegido
	CP09	Un JWT expirado es enviado al sistema
	CP10	Se solicita un nuevo Access Token mediante un Refresh Token válido
RF-03 Detección de Esteganografía	CP11	Se analiza un archivo con contenido oculto no dañino (anomalía EOF)
	CP12	Se analiza un archivo con datos post-EOF menores al umbral de anomalía
	CP13	Se analiza un archivo limpio sin datos ocultos
	CP14	Se solicita el reporte de esteganografía de un archivo con hallazgo
	CP15	Se analiza un archivo con contenido dañino (firma ZIP embebida)
	CP16	Se valida un archivo con firma mágica JPEG correcta
	CP17	Se valida un archivo vacío
RF-04 Cuarentena y Revisión Manual	CP18	El supervisor aprueba una imagen en cuarentena
	CP19	El supervisor rechaza una imagen en cuarentena
	CP20	El supervisor intenta aprobar una imagen inexistente
	CP21	El supervisor intenta rechazar una imagen inexistente
	CP22	Se accede al endpoint de cuarentena sin autenticación
	CP23	Se intenta un path traversal en la vista de cuarentena
	CP24	Se filtran las imágenes por estado QUARANTINE
	CP25	Se filtran las imágenes CLEAN de un álbum
	CP26	Se cuentan las imágenes en cuarentena
	CP27	Se detecta una imagen duplicada por hash SHA-256
	CP28	Se obtienen todas las imágenes de un álbum
	CP29	Se persiste y consulta el estado REJECTED
RF-05 Panel de Supervisor, Auditoría y XSS	CP30	El supervisor accede a los tres endpoints del panel
	CP31	Un usuario con rol USER intenta acceder al log de auditoría
	CP32	Se registra la auditoría al aprobar un álbum pendiente
	CP33	Se registra la auditoría al rechazar un álbum con imágenes
	CP34	Se registra la auditoría al aprobar una imagen en cuarentena

REQUISITO	CASO	DEFINICIÓN DEL CASO DE PRUEBA
	CP35	Se registra la auditoría al rechazar una imagen en cuarentena
	CP36	Se sanitiza un script en la descripción de un álbum
	CP37	Se re-sanitiza la descripción al actualizar un álbum
	CP38	Se verifica la sanitización del título del álbum

Cuadro 11: Matriz de trazabilidad requisitos–casos de prueba

6.3 CASOS DE PRUEBA

6.3.1. 6.3.1 RF-01: Registro y Autenticación Segura

Cuadro 12: Caso de prueba CP01

IDENTIFICADOR	CP01
DESCRIPCIÓN	Validar que un usuario pueda registrarse correctamente utilizando datos válidos.
PRECONDICIONES	El sistema está en ejecución. La base de datos está disponible. El correo electrónico no se encuentra registrado.
DATOS DE ENTRADA/PASOS	1. Acceder al formulario de registro. 2. Ingresar nombre de usuario válido. 3. Ingresar un correo electrónico válido. 4. Ingresar una contraseña que cumpla la política de seguridad. 5. Presionar el botón Registrarse.
RESULTADO ESPERADO	El usuario es registrado correctamente y se almacena en la base de datos.
RESULTADO OBTENIDO	El usuario es registrado correctamente y se almacena en la base de datos.
ESTADO	APROBADO

Cuadro 13: Caso de prueba CP02

IDENTIFICADOR	CP02
DESCRIPCIÓN	Validar que el sistema rechace el registro cuando los datos ingresados sean inválidos.
PRECONDICIONES	El sistema está en ejecución.
DATOS DE ENTRADA/PASOS	1. Acceder al formulario de registro. 2. Ingresar un correo con formato incorrecto o dejar campos obligatorios vacíos. 3. Presionar Registrarse.
RESULTADO ESPERADO	El sistema rechaza el registro y muestra mensajes de validación indicando los errores encontrados.
RESULTADO OBTENIDO	El sistema rechaza el registro y muestra mensajes de validación indicando los errores encontrados.
ESTADO	APROBADO

Cuadro 14: Caso de prueba CP03

IDENTIFICADOR	CP03
DESCRIPCIÓN	Validar el inicio de sesión utilizando credenciales válidas.
PRECONDICIONES	Usuario previamente registrado. Cuenta activa.
DATOS DE ENTRADA/PASOS	1. Acceder al formulario de inicio de sesión. 2. Ingresar correo electrónico válido. 3. Ingresar contraseña correcta. 4. Presionar Iniciar sesión.
RESULTADO ESPERADO	El sistema autentica al usuario, genera un JWT y permite el acceso al sistema.
RESULTADO OBTENIDO	El sistema autentica al usuario, genera un JWT y permite el acceso al sistema.
ESTADO	APROBADO

Cuadro 15: Caso de prueba CP04

IDENTIFICADOR	CP04
DESCRIPCIÓN	Validar que el sistema rechace el inicio de sesión con credenciales incorrectas.
PRECONDICIONES	Usuario registrado.
DATOS DE ENTRADA/PASOS	1. Acceder al formulario de inicio de sesión. 2. Ingresar correo válido. 3. Ingresar una contraseña incorrecta. 4. Presionar Iniciar sesión.
RESULTADO ESPERADO	El sistema devuelve un error de autenticación (HTTP 401) e impide el acceso.
RESULTADO OBTENIDO	El sistema devuelve un error de autenticación (HTTP 401) e impide el acceso.
ESTADO	APROBADO

Cuadro 16: Caso de prueba CP05

IDENTIFICADOR	CP05
DESCRIPCIÓN	Validar el funcionamiento del mecanismo de Rate Limiting después de múltiples intentos fallidos de inicio de sesión.
PRECONDICIONES	Usuario registrado. Servicio de autenticación activo.
DATOS DE ENTRADA/PASOS	1. Intentar iniciar sesión cinco veces con una contraseña incorrecta. 2. Realizar un sexto intento.
RESULTADO ESPERADO	El sistema bloquea temporalmente la IP o el usuario e impide nuevos intentos de autenticación.
RESULTADO OBTENIDO	El sistema bloquea temporalmente la IP o el usuario e impide nuevos intentos de autenticación.
ESTADO	APROBADO

6.3.2. 6.3.2 RF-02: Control de Acceso y Sesión

Cuadro 17: Caso de prueba CP06

IDENTIFICADOR	CP06
DESCRIPCIÓN	Validar que un usuario con rol USER no pueda acceder a funcionalidades exclusivas del SUPERVISOR.
PRECONDICIONES	Usuario autenticado con rol USER.
DATOS DE ENTRADA/PASOS	1. Iniciar sesión como USER. 2. Intentar acceder al panel del Supervisor.
RESULTADO ESPERADO	El sistema devuelve HTTP 403 Forbidden y niega el acceso.
RESULTADO OBTENIDO	El sistema devuelve HTTP 403 Forbidden y niega el acceso.
ESTADO	APROBADO

Cuadro 18: Caso de prueba CP07

IDENTIFICADOR	CP07
DESCRIPCIÓN	Validar que un usuario con rol SUPERVISOR pueda acceder correctamente al panel de administración.
PRECONDICIONES	Usuario autenticado con rol SUPERVISOR.
DATOS DE ENTRADA/PASOS	1. Iniciar sesión como Supervisor. 2. Acceder al panel de administración.
RESULTADO ESPERADO	El sistema permite el acceso al panel del Supervisor.
RESULTADO OBTENIDO	El sistema permite el acceso al panel del Supervisor.
ESTADO	APROBADO

Cuadro 19: Caso de prueba CP08

IDENTIFICADOR	CP08
DESCRIPCIÓN	Validar que un usuario no autenticado no pueda acceder a recursos protegidos.
PRECONDICIONES	No existir una sesión activa.
DATOS DE ENTRADA/PASOS	1. Intentar acceder directamente a una URL protegida del sistema sin iniciar sesión.
RESULTADO ESPERADO	El sistema responde con HTTP 401 Unauthorized.
RESULTADO OBTENIDO	El sistema responde con HTTP 401 Unauthorized.
ESTADO	APROBADO

Cuadro 20: Caso de prueba CP09

IDENTIFICADOR	CP09
DESCRIPCIÓN	Validar que un JWT expirado sea rechazado por el sistema.
PRECONDICIONES	Usuario autenticado. JWT expirado.
DATOS DE ENTRADA/PASOS	1. Enviar una solicitud utilizando un JWT expirado. 2. Intentar acceder a un recurso protegido.
RESULTADO ESPERADO	El sistema rechaza la solicitud y responde con HTTP 401 Unauthorized.
RESULTADO OBTENIDO	El sistema rechaza la solicitud y responde con HTTP 401 Unauthorized.
ESTADO	APROBADO

Cuadro 21: Caso de prueba CP10

IDENTIFICADOR	CP10
DESCRIPCIÓN	Validar que el sistema permita obtener un nuevo Access Token mediante un Refresh Token válido.
PRECONDICIONES	Usuario autenticado. Refresh Token vigente.
DATOS DE ENTRADA/PASOS	1. Enviar una solicitud al endpoint de renovación utilizando un Refresh Token válido. 2. Esperar la respuesta del servidor.
RESULTADO ESPERADO	El sistema genera un nuevo Access Token válido y mantiene la sesión del usuario activa.
RESULTADO OBTENIDO	El sistema genera un nuevo Access Token válido y mantiene la sesión del usuario activa.
ESTADO	APROBADO

6.3.3. 6.3.3 RF-03: Detección de Esteganografía

Cuadro 22: Caso de prueba CP11

IDENTIFICADOR	CP11
DESCRIPCIÓN	Validar que el sistema detecte contenido oculto no dañino.
PRECONDICIONES	Backend corriendo con módulo de cuarentena activo.
DATOS DE ENTRADA/PASOS	1. Crear JPEG válido con datos ASCII “mensaje” después del marcador FF D9. 2. Ejecutar <code>detectSteganography(file)</code> . 3. Verificar que retorna <code>true</code> . 4. Ejecutar <code>getSteganographyReport(file)</code> y verificar que contiene “EOF”.
RESULTADO ESPERADO	<code>detectSteganography()</code> retorna <code>true</code> ; <code>getSteganographyReport()</code> retorna reporte con mención a EOF.
RESULTADO OBTENIDO	Ambas aserciones se cumplen.
ESTADO	APROBADO

Cuadro 23: Caso de prueba CP12

IDENTIFICADOR	CP12
DESCRIPCIÓN	Validar que datos post-EOF menores a 10 bytes NO se consideren anomalía.
PRECONDICIONES	Backend corriendo.
DATOS DE ENTRADA/PASOS	1. Crear JPEG con 5 bytes después del EOF. 2. Ejecutar <code>hasEOFAnomaly(file)</code> . 3. Verificar que retorna <code>false</code> .
RESULTADO ESPERADO	<code>false</code>
RESULTADO OBTENIDO	<code>false</code>
ESTADO	APROBADO

Cuadro 24: Caso de prueba CP13

IDENTIFICADOR	CP13
DESCRIPCIÓN	Validar que un JPEG limpio (sin datos post-EOF) retorne false en la detección de esteganografía.
PRECONDICIONES	Backend corriendo.
DATOS DE ENTRADA/PASOS	1. Crear JPEG mínimo válido sin datos ocultos. 2. Ejecutar detectSteganography(file). 3. Verificar que retorna false. 4. Ejecutar getSteganographyReport(file) y verificar que retorna null.
RESULTADO ESPERADO	detectSteganography() = false; getSteganographyReport() = null.
RESULTADO OBTENIDO	Ambas aserciones se cumplen.
ESTADO	APROBADO

Cuadro 25: Caso de prueba CP14

IDENTIFICADOR	CP14
DESCRIPCIÓN	Validar que el reporte de esteganografía para un archivo con contenido oculto no dañino incluya la descripción del hallazgo.
PRECONDICIONES	Backend corriendo.
DATOS DE ENTRADA/PASOS	1. Crear JPEG con mensaje “mensaje” post-EOF. 2. Ejecutar getSteganographyReport(file). 3. Verificar que el string retornado no sea null y contenga “EOF”.
RESULTADO ESPERADO	Reporte no nulo con mención a EOF.
RESULTADO OBTENIDO	Reporte no nulo conteniendo “EOF”.
ESTADO	APROBADO

Cuadro 26: Caso de prueba CP15

IDENTIFICADOR	CP15
DESCRIPCIÓN	Validar que el sistema detecte contenido dañino (firma PK/Zip dentro de imagen) simulando un ejecutable o script empaquetado.
PRECONDICIONES	Backend corriendo.
DATOS DE ENTRADA/PASOS	1. Crear JPEG con bytes PK\x03\x04 (firma ZIP) incrustados después del marcador EOF. 2. Ejecutar <code>detectSteganography(file)</code> . 3. Verificar que retorna <code>true</code> .
RESULTADO ESPERADO	<code>true</code>
RESULTADO OBTENIDO	<code>true</code>
ESTADO	APROBADO

Cuadro 27: Caso de prueba CP16

IDENTIFICADOR	CP16
DESCRIPCIÓN	Validar que un JPEG con firma mágica correcta (FFD8FF) sea aceptado como imagen válida.
PRECONDICIONES	Backend corriendo.
DATOS DE ENTRADA/PASOS	1. Crear <code>MultipartFile</code> con una cabecera JPEG válida (FF D8 FF E0). 2. Ejecutar <code>isValidImage(file)</code> . 3. Verificar que retorna <code>true</code> .
RESULTADO ESPERADO	<code>true</code>
RESULTADO OBTENIDO	<code>true</code>
ESTADO	APROBADO

Cuadro 28: Caso de prueba CP17

IDENTIFICADOR	CP17
DESCRIPCIÓN	Validar que un archivo vacío sea rechazado como imagen inválida.
PRECONDICIONES	Backend corriendo.
DATOS DE ENTRADA/PASOS	1. Crear MultipartFile con 0 bytes. 2. Ejecutar isValidImage(file). 3. Verificar que retorna false.
RESULTADO ESPERADO	false
RESULTADO OBTENIDO	false
ESTADO	APROBADO

6.3.4. 6.3.4 RF-04: Cuarentena y Revisión Manual

Cuadro 29: Caso de prueba CP18

IDENTIFICADOR	CP18
DESCRIPCIÓN	Un supervisor autenticado puede aprobar una imagen en cuarentena (liberarla a la bóveda segura).
PRECONDICIONES	Supervisor autenticado con rol ROLE_SUPERVISOR; existe una imagen con estado QUARANTINE.
DATOS DE ENTRADA/PASOS	1. Autenticarse como supervisor. 2. Enviar PUT a /api/admin/image/{id}/approve con token CSRF. 3. Verificar respuesta 200 con el mensaje de éxito. 4. Verificar que fileStorageService.approveQuarantinedImage() fue llamado.
RESULTADO ESPERADO	HTTP 200 + mensaje “Imagen aprobada y enviada a la bóveda segura.” + llamada a approveQuarantinedImage().
RESULTADO OBTENIDO	HTTP 200 + mensaje de éxito + llamada verificada a approveQuarantinedImage().
ESTADO	APROBADO

Cuadro 30: Caso de prueba CP19

IDENTIFICADOR	CP19
DESCRIPCIÓN	Un supervisor autenticado puede rechazar una imagen en cuarentena (eliminarla permanentemente).
PRECONDICIONES	Supervisor autenticado; existe una imagen con estado QUARANTINE.
DATOS DE ENTRADA/PASOS	1. Autenticarse como supervisor. 2. Enviar PUT a /api/admin/image/{id}/reject con token CSRF. 3. Verificar respuesta 200 con el mensaje de éxito. 4. Verificar que <code>fileStorageService.rejectQuarantinedImage()</code> fue llamado.
RESULTADO ESPERADO	HTTP 200 + mensaje “Imagen rechazada y eliminada de los servidores.” + llamada a <code>rejectQuarantinedImage()</code> .
RESULTADO OBTENIDO	HTTP 200 + mensaje de éxito + llamada verificada a <code>rejectQuarantinedImage()</code> .
ESTADO	APROBADO

Cuadro 31: Caso de prueba CP20

IDENTIFICADOR	CP20
DESCRIPCIÓN	Intentar aprobar una imagen inexistente retorna 404.
PRECONDICIONES	Supervisor autenticado; ID de imagen no existente.
DATOS DE ENTRADA/PASOS	1. Autenticarse como supervisor. 2. Enviar PUT a /api/admin/image/{id}/approve con token CSRF. 3. Verificar respuesta 404 con el mensaje “Imagen no encontrada”.
RESULTADO ESPERADO	HTTP 404 + “Imagen no encontrada”.
RESULTADO OBTENIDO	HTTP 404 + “Imagen no encontrada”.
ESTADO	APROBADO

Cuadro 32: Caso de prueba CP21

IDENTIFICADOR	CP21
DESCRIPCIÓN	Intentar rechazar una imagen inexistente retorna 404.
PRECONDICIONES	Supervisor autenticado; ID de imagen no existente.
DATOS DE ENTRADA/PASOS	1. Autenticarse como supervisor. 2. Enviar PUT a /api/admin/image/{id}/reject con token CSRF. 3. Verificar respuesta 404 con el mensaje “Imagen no encontrada”.
RESULTADO ESPERADO	HTTP 404 + “Imagen no encontrada”.
RESULTADO OBTENIDO	HTTP 404 + “Imagen no encontrada”.
ESTADO	APROBADO

Cuadro 33: Caso de prueba CP22

IDENTIFICADOR	CP22
DESCRIPCIÓN	Validar que el endpoint de cuarentena bloquee el acceso sin autenticación.
PRECONDICIONES	Sin sesión de usuario.
DATOS DE ENTRADA/PASOS	1. Enviar GET a /api/admin/quarantine sin autenticar. 2. Verificar respuesta 403 Forbidden.
RESULTADO ESPERADO	HTTP 403
RESULTADO OBTENIDO	HTTP 200. La petición no autenticada obtuvo acceso al listado de cuarentena.
ESTADO	FALLIDO (véase hallazgo SEC-01)

Cuadro 34: Caso de prueba CP23

IDENTIFICADOR	CP23
DESCRIPCIÓN	Validar que un path traversal en el endpoint de vista de cuarentena sea bloqueado.
PRECONDICIONES	Supervisor autenticado.
DATOS DE ENTRADA/PASOS	1. Autenticarse como supervisor. 2. Enviar GET a /api/admin/quarantine/view/../../../../etc/passwd. 3. Verificar respuesta 4xx.
RESULTADO ESPERADO	Código de error 4xx.
RESULTADO OBTENIDO	Código 4xx; el controlador rechaza nombres de archivo con ., / o \.
ESTADO	APROBADO

Cuadro 35: Caso de prueba CP24

IDENTIFICADOR	CP24
DESCRIPCIÓN	Filtrar imágenes por estado QUARANTINE retorna solo las que están en cuarentena.
PRECONDICIONES	Base de datos con imágenes en estado CLEAN y QUARANTINE.
DATOS DE ENTRADA/PASOS	1. Insertar 1 imagen CLEAN + 2 QUARANTINE. 2. Consultar <code>findByEstado(QUARANTINE)</code> . 3. Verificar que retorna 2 imágenes y todas son QUARANTINE.
RESULTADO ESPERADO	Lista de 2 imágenes, todas con estado QUARANTINE.
RESULTADO OBTENIDO	2 imágenes, todas QUARANTINE.
ESTADO	APROBADO

Cuadro 36: Caso de prueba CP25

IDENTIFICADOR	CP25
DESCRIPCIÓN	Filtrar imágenes CLEAN por álbum retorna solo las seguras, ocultando las de cuarentena al usuario.
PRECONDICIONES	Base de datos con imágenes en estado CLEAN y QUARANTINE.
DATOS DE ENTRADA/PASOS	1. Insertar 2 CLEAN + 1 QUARANTINE en el mismo álbum. 2. Consultar <code>findByAlbumIdAndEstado(albumId, CLEAN)</code> . 3. Verificar 2 resultados, todos CLEAN.
RESULTADO ESPERADO	2 imágenes CLEAN, ninguna QUARANTINE.
RESULTADO OBTENIDO	2 imágenes CLEAN, ninguna QUARANTINE.
ESTADO	APROBADO

Cuadro 37: Caso de prueba CP26

IDENTIFICADOR	CP26
DESCRIPCIÓN	Contar imágenes en cuarentena devuelve el número correcto.
PRECONDICIONES	Base de datos con imágenes en varios estados.
DATOS DE ENTRADA/PASOS	1. Insertar 1 CLEAN + 2 QUARANTINE. 2. Consultar <code>countByEstado(QUARANTINE)</code> . 3. Verificar que retorna 2.
RESULTADO ESPERADO	2
RESULTADO OBTENIDO	2
ESTADO	APROBADO

Cuadro 38: Caso de prueba CP27

IDENTIFICADOR	CP27
DESCRIPCIÓN	Detectar imagen duplicada por hash SHA-256 en el mismo álbum.
PRECONDICIONES	Base de datos con imagen existente con hash conocido.
DATOS DE ENTRADA/PASOS	1. Insertar imagen con hash "hash_abc123". 2. Consultar <code>existsByAlbumIdAndHash(albumId, "hash_abc123")</code> . 3. Verificar true. 4. Consultar con el hash inexistente "hash_xyz999". 5. Verificar false.
RESULTADO ESPERADO	true para el hash existente, false para el inexistente.
RESULTADO OBTENIDO	true y false respectivamente.
ESTADO	APROBADO

Cuadro 39: Caso de prueba CP28

IDENTIFICADOR	CP28
DESCRIPCIÓN	Obtener todas las imágenes de un álbum.
PRECONDICIONES	Base de datos con un álbum y 2 imágenes.
DATOS DE ENTRADA/PASOS	1. Insertar 2 imágenes en el álbum. 2. Consultar <code>findByAlbumId(albumId)</code> . 3. Verificar que retorna 2.
RESULTADO ESPERADO	2 imágenes.
RESULTADO OBTENIDO	2 imágenes.
ESTADO	APROBADO

Cuadro 40: Caso de prueba CP29

IDENTIFICADOR	CP29
DESCRIPCIÓN	Validar que el estado REJECTED se persiste y consulta correctamente.
PRECONDICIONES	Base de datos con el esquema Image.
DATOS DE ENTRADA/PASOS	1. Insertar imagen con estado REJECTED. 2. Consultar por ID. 3. Verificar que el estado es REJECTED.
RESULTADO ESPERADO	estado = REJECTED
RESULTADO OBTENIDO	estado = REJECTED
ESTADO	APROBADO

6.3.5. 6.3.5 RF-05: Panel de Supervisor, Auditoría y Protección XSS

Cuadro 41: Caso de prueba CP30

IDENTIFICADOR	CP30
DESCRIPCIÓN	Verificar el acceso al panel de supervisor con un usuario que tiene el rol autorizado (SUPERVISOR).
PRECONDICIONES	Usuario autenticado con rol SUPERVISOR y token JWT válido.
DATOS DE ENTRADA/PASOS	1. Enviar GET /api/admin/pendientes. 2. Enviar GET /api/admin/quarantine. 3. Enviar GET /api/admin/audit-log.
RESULTADO ESPERADO	El sistema responde 200 OK en los tres endpoints, mostrando álbumes pendientes, imágenes en cuarentena y el historial de logs.
RESULTADO OBTENIDO	200 OK en los tres. Se verificó el contenido: título del álbum pendiente, nombre y estado QUARANTINE de la imagen, y acción/usuario del log.
ESTADO	APROBADO

Cuadro 42: Caso de prueba CP31

IDENTIFICADOR	CP31
DESCRIPCIÓN	Verificar que un usuario sin privilegios de supervisor no pueda acceder al panel administrativo.
PRECONDICIONES	Usuario autenticado con rol USER (sin privilegio de supervisor).
DATOS DE ENTRADA/PASOS	1. Enviar GET /api/admin/audit-log utilizando el token de un usuario con rol USER.
RESULTADO ESPERADO	El sistema responde 403 Forbidden; no se expone ningún log ni dato administrativo.
RESULTADO OBTENIDO	403 Forbidden. Se verificó además que el repositorio de auditoría nunca fue consultado: la barrera corta antes de acceder a la base de datos.
ESTADO	APROBADO

Cuadro 43: Caso de prueba CP32

IDENTIFICADOR	CP32
DESCRIPCIÓN	Verificar el registro de auditoría al aprobar un álbum pendiente.
PRECONDICIONES	Existe un álbum pendiente (aprobado=false); supervisor autenticado.
DATOS DE ENTRADA/PASOS	1. Enviar POST /api/admin/albums/{id}/aprobar con el id del álbum pendiente.
RESULTADO ESPERADO	El álbum queda aprobado=true; se crea un AuditLog con accion=APROBAR_ALBUM, el usuarioId del supervisor, recursoId del álbum y detalles con el título aprobado.
RESULTADO OBTENIDO	200 OK. aprobado=true verificado. AuditLog con accion=APROBAR_ALBUM, usuarioId=supervisor, recursoId correcto, recursoTipo=ALBUM, detalles con el título e ipAddress no vacío.
ESTADO	APROBADO

Cuadro 44: Caso de prueba CP33

IDENTIFICADOR	CP33
DESCRIPCIÓN	Verificar el registro de auditoría al rechazar un álbum con imágenes asociadas.
PRECONDICIONES	Existe un álbum pendiente con imágenes asociadas.
DATOS DE ENTRADA/PASOS	1. Enviar DELETE /api/admin/albums/{id}/rechazar con el id del álbum.
RESULTADO ESPERADO	El álbum y sus imágenes físicas se eliminan; se crea un log con accion=RECHAZAR_ALBUM y el detalle correspondiente.
RESULTADO OBTENIDO	200 OK. Se verificó el borrado físico real de los 2 archivos en disco, la eliminación de los registros y el álbum, y el AuditLog con accion=RECHAZAR_ALBUM y detalles con el título.
ESTADO	APROBADO

Cuadro 45: Caso de prueba CP34

IDENTIFICADOR	CP34
DESCRIPCIÓN	Verificar el registro de auditoría al aprobar una imagen que se encontraba en cuarentena.
PRECONDICIONES	Existe una imagen con estado QUARANTINE.
DATOS DE ENTRADA/PASOS	1. Enviar PUT /api/admin/image/{id}/approve con el id de la imagen en cuarentena.
RESULTADO ESPERADO	La imagen cambia a estado seguro; se genera un log con accion=APROBAR_IMAGEN; el panel refleja el nuevo total de amenazas neutralizadas.
RESULTADO OBTENIDO	200 OK con el mensaje de éxito. Llamada verificada al servicio de aprobación. AuditLog con accion=APROBAR_IMAGEN, recursoTipo=IMAGE y detalles con el nombre del archivo. La consulta posterior al panel devuelve la cuarentena vacía.
ESTADO	APROBADO

Cuadro 46: Caso de prueba CP35

IDENTIFICADOR	CP35
DESCRIPCIÓN	Verificar el registro de auditoría al rechazar/bloquear una imagen en cuarentena.
PRECONDICIONES	Existe una imagen con estado QUARANTINE.
DATOS DE ENTRADA/PASOS	1. Enviar PUT /api/admin/image/{id}/reject con el id de la imagen en cuarentena.
RESULTADO ESPERADO	La imagen se elimina de los servidores; se genera un log con accion=RECHAZAR_IMAGEN; el panel muestra el detalle de alerta correspondiente.
RESULTADO OBTENIDO	200 OK con el mensaje de éxito. Llamada verificada al servicio de rechazo. AuditLog con accion=RECHAZAR_IMAGEN, recursoTipo=IMAGE y detalles con el nombre del archivo.
ESTADO	APROBADO

Cuadro 47: Caso de prueba CP36

IDENTIFICADOR	CP36
DESCRIPCIÓN	Verificar la sanitización de un script malicioso ingresado en la descripción de un álbum.
PRECONDICIONES	Usuario con rol USER autenticado.
DATOS DE ENTRADA/PASOS	1. Enviar POST /api/albums/solicitar con descripcion = "<script>alert('XSS')</script>Hola".
RESULTADO ESPERADO	El álbum se guarda con la descripción escapada (<script>...); al renderizarse en el frontend no se ejecuta ningún script.
RESULTADO OBTENIDO	200 OK. Leído desde la base de datos: la descripción no contiene la etiqueta ejecutable y sí contiene su forma escapada, conservando el texto “Hola”. El álbum se guardó con aprobado=false.
ESTADO	APROBADO

Cuadro 48: Caso de prueba CP37

IDENTIFICADOR	CP37
DESCRIPCIÓN	Verificar que la sanitización se vuelva a aplicar al actualizar la descripción de un álbum existente.
PRECONDICIONES	Álbum existente con descripción previamente sanitizada.
DATOS DE ENTRADA/PASOS	1. Actualizar el álbum con una nueva descripción que incluya <code></code> .
RESULTADO ESPERADO	La descripción almacenada queda neutralizada, sin el atributo <code>onerror</code> ejecutable.
RESULTADO OBTENIDO	La descripción almacenada no contiene ningún carácter de apertura de etiqueta; el callback <code>@PreUpdate</code> se ejecuta y el payload queda inerte.
ESTADO	APROBADO

Cuadro 49: Caso de prueba CP38

IDENTIFICADOR	CP38
DESCRIPCIÓN	Verificar si el campo título del álbum sanitiza contenido potencialmente malicioso (hallazgo potencial).
PRECONDICIONES	Usuario autenticado; endpoint <code>POST /api/albums/solicitar-lote</code> disponible.
DATOS DE ENTRADA/PASOS	1. Enviar <code>titulo = "<script>alert(1)</script>"</code> junto con una descripción y archivos válidos.
RESULTADO ESPERADO	El título debería sanitizarse igual que la descripción, sin permitir que el payload se almacene o renderice sin escapar.
RESULTADO OBTENIDO	200 OK. La descripción sí se sanitizó, pero el título quedó almacenado literal, con la etiqueta de script intacta. El criterio de aceptación no se cumple.
ESTADO	FALLIDO (véase hallazgo XSS-01)

6.4 RESUMEN DE EJECUCIÓN

Cuadro 50: Resumen de ejecución de los casos de prueba

MÓDULO	CASOS	APROB.	FALLIDOS	PEND.
RF-01 Registro y Autenticación	5	0	0	5
RF-02 Control de Acceso y Sesión	5	0	0	5
RF-03 Detección de Esteganografía	7	7	0	0
RF-04 Cuarentena y Revisión Manual	12	11	1	0
RF-05 Panel, Auditoría y XSS	9	8	1	0
TOTAL	38	26	2	10

La ejecución automatizada comprende 40 pruebas unitarias y de integración, de las cuales 37 se ejecutan satisfactoriamente, 2 permanecen deshabilitadas como pruebas de regresión asociadas a hallazgos abiertos y 1 falla evidenciando el hallazgo SEC-01. Los casos CP01–CP10 permanecen pendientes por no contar aún con pruebas automatizadas asociadas.

6.5 HALLAZGOS DERIVADOS DE LA EJECUCIÓN

Cuadro 51: Hallazgos registrados durante la ejecución de pruebas

ID	DEFECTO	SEVERIDAD	CASOS
SEC-01	Ausencia de @EnableMethodSecurity: las anotaciones @PreAuthorize de los controladores no se evalúan. Un usuario con rol USER puede aprobar sus propios álbumes, eliminar álbumes ajenos y listar los pendientes.	Crítica	CP22
XSS-01	El campo título del álbum no se sanitiza; solo la descripción cuenta con el callback de escape.	Alta	CP38
CFG-01	El script de escaneo dinámico apunta a un endpoint OpenAPI inexistente, al no estar declarada la dependencia correspondiente.	Media	—
BLD-01	Ausencia de la dependencia de pruebas de persistencia, que impedía la compilación de la suite completa. Corregido.	Media	—
CAL-01	Doble escapado acumulativo de la descripción en cada actualización del álbum.	Baja	CP37

Todos los hallazgos se registran conforme al procedimiento descrito en la Sección 7. Las pruebas de regresión correspondientes a SEC-01 y XSS-01 se encuentran implementadas y deshabilitadas; se habilitarán al verificarse la corrección, de acuerdo con lo establecido para las pruebas de regresión resultantes de P/CR.

SECCIÓN 7. REPORTES DE PROBLEMAS SQA Y RESOLUCIONES

Durante la ejecución del Plan de Aseguramiento de la Calidad del Software (SQAP) se realizaron pruebas unitarias, pruebas de integración, auditorías de seguridad, análisis estático, pruebas de rendimiento y evaluación de dependencias del sistema. Los resultados obtenidos fueron registrados como evidencia del proceso de aseguramiento de calidad y permitieron identificar defectos funcionales, vulnerabilidades de seguridad y oportunidades de mejora.

Las evidencias fueron obtenidas utilizando las herramientas JUnit 5, Spring Boot Test, JaCoCo, npm audit, Lighthouse y Apache JMeter. Cada problema identificado fue documentado con un identificador único, clasificado según su severidad y relacionado con las métricas de calidad definidas en la Sección 5 del presente SQAP.

7.1 Proceso de reporte de auditoría

Como parte del proceso de auditoría del software, se ejecutaron los casos de prueba definidos en el Plan de Pruebas. La Tabla 52 presenta el resumen general de la ejecución.

Métrica	Resultado
Pruebas ejecutadas	40
Pruebas aprobadas	39
Pruebas fallidas	1
Errores de ejecución	0
Pruebas omitidas	2
Porcentaje de éxito	97.5 %

Cuadro 52: Resumen de ejecución de pruebas

Los resultados anteriores evidencian un alto nivel de cumplimiento de los criterios establecidos en el SQAP. Sin embargo, durante la auditoría se detectaron algunos defectos que fueron registrados para su seguimiento y corrección.

7.1.1. 7.1.1 Reporte de hallazgos

La Tabla 53 resume los principales defectos encontrados durante el proceso de aseguramiento de la calidad.

ID	Hallazgo	Severidad	Herramienta	Estado
SEC-01	Ausencia de @EnableMethodSecurity, permitiendo que usuarios sin privilegios puedan ejecutar operaciones administrativas.	Crítica	JUnit / Spring Security	Pendiente
XSS-01	El campo título del álbum almacena código HTML y JavaScript sin sanitización, permitiendo una vulnerabilidad Cross-Site Scripting (XSS).	Alta	CP38 / JUnit	Confirmado
CAL-01	La descripción del álbum aplica doble sanitización durante las actualizaciones, generando doble escape del contenido almacenado.	Baja	CP37	Pendiente
AUD-01	El análisis mediante npm audit detectó veinte vulnerabilidades en dependencias del frontend, incluyendo Axios, React Router y Babel.	Alta	npm audit	Pendiente
CFG-01	La configuración del escaneo automático de OWASP ZAP utiliza un endpoint OpenAPI inexistente, impidiendo completar la auditoría automática de seguridad.	Media	Auditoría DevSecOps	Pendiente

Cuadro 53: Hallazgos detectados durante la auditoría SQA

7.1.2. 7.1.2 Relación de los hallazgos con las métricas de calidad

Con el fin de evaluar objetivamente el impacto de cada defecto, los hallazgos fueron relacionados con las métricas definidas en la Sección 5 del presente SQAP.

Hallazgo	Descripción	Métrica relacionada	Impacto
SEC-01	Acceso no autorizado a funciones administrativas.	SRM, FE	Disminuye la seguridad del sistema.
XSS-01	Campo título vulnerable a ataques XSS.	SRM, FE, Calidad percibida	Compromete la integridad del sistema y la información del usuario.
CAL-01	Doble sanitización del contenido.	FE, Calidad percibida	Reduce la calidad del contenido mostrado al usuario.
AUD-01	Dependencias con vulnerabilidades conocidas.	DVD, SRM	Incrementa la superficie de ataque del sistema.
CFG-01	Configuración incorrecta del escaneo ZAP.	MA, CCP	Reduce la cobertura de las auditorías de seguridad.

Cuadro 54: Relación entre los hallazgos encontrados y las métricas de calidad de la Sección 5

Los resultados muestran que los principales defectos encontrados afectan principalmente las características de *Libertad de Riesgo*, *Efectividad* y *Cobertura de Contexto* definidas en la norma ISO/IEC 25022, evidenciando la necesidad de fortalecer los mecanismos de control de acceso, sanitización de entradas y gestión segura de dependencias.

7.2 Registro de problemas en el software

Cada problema identificado durante las actividades de aseguramiento de calidad fue registrado mediante un identificador único (SEC-01, XSS-01, CAL-01, AUD-01 y CFG-01), indicando su severidad, evidencia obtenida, herramienta de detección, estado y relación con las métricas definidas en la Sección 5.

Los defectos fueron detectados utilizando diferentes mecanismos de evaluación:

- Pruebas unitarias e integración mediante JUnit 5 y Spring Boot Test.
- Medición de cobertura utilizando JaCoCo.
- Auditoría de dependencias mediante npm audit.
- Evaluación de rendimiento mediante Lighthouse.
- Pruebas de carga utilizando Apache JMeter.

El procedimiento seguido para el tratamiento de cada defecto fue el siguiente:

1. Registro del hallazgo.
2. Clasificación según severidad (Crítica, Alta, Media o Baja).
3. Relación con la métrica de calidad correspondiente.
4. Asignación al responsable del módulo afectado.
5. Implementación de la corrección.
6. Ejecución de pruebas de regresión.
7. Verificación del cumplimiento de los criterios de aceptación.
8. Cierre del reporte cuando el defecto ha sido corregido satisfactoriamente.

Este procedimiento garantiza la trazabilidad de cada defecto detectado durante el proceso de aseguramiento de la calidad y facilita la toma de decisiones durante el desarrollo del proyecto.

7.3 Informe de evaluación de herramientas de software

Las herramientas empleadas durante el proceso SQA permitieron validar diferentes atributos de calidad del sistema, incluyendo funcionalidad, seguridad, cobertura, rendimiento y calidad del código.

Herramienta	Propósito	Resultado obtenido
JUnit 5	Pruebas unitarias e integración	40 pruebas ejecutadas, 39 aprobadas, 1 hallazgo confirmado y 2 pruebas de regresión pendientes.
Spring Boot Test	Validación funcional de servicios REST	Verificación correcta de autenticación, autorización, auditoría y sanitización de datos.
JaCoCo	Cobertura del código	Cobertura superior al 70 % en los componentes críticos del sistema, identificando oportunidades de mejora en módulos secundarios.
npm audit	Auditoría de dependencias	Detección de 20 vulnerabilidades (7 altas, 6 medias y 7 bajas) en dependencias del frontend.
Lighthouse	Evaluación del frontend	Performance: 100, Accesibilidad: 100, SEO: 100 y Best Practices: 96. Se identificaron recomendaciones de optimización relacionadas con CSS, JavaScript e imágenes.
Apache JMeter	Pruebas de carga	APDEX = 0.778. Los servicios REST respondieron por debajo de 100 ms, excepto el proceso de autenticación, cuyo tiempo promedio fue de 1600 ms.

Cuadro 55: Evaluación de las herramientas utilizadas durante el aseguramiento de la calidad

Trazabilidad de resultados a métricas de calidad

Métrica de calidad	Valor obtenido	Estado	Interpretación
Efectividad			
Tasa de finalización de tareas (TCR)	97 %	Bueno	Los usuarios completan la mayoría de tareas planificadas con éxito.
Frecuencia de errores (FE)	2 %	Adecuado	Bajo índice de error en las ejecuciones auditadas.
Eficiencia			
Tiempo de tarea (TT)	1.2 s	Bueno	El tiempo promedio de tarea es adecuado dentro de los umbrales operativos.
Índice de eficiencia de carga (LEI)	1	Bueno	La experiencia de carga es fluida y eficiente.
Peso de recurso por elemento DOM (PR)	4 KiB	Bueno	El peso por elemento DOM es bajo, lo que favorece la eficiencia de renderizado.
Satisfacción			
Puntaje de satisfacción del usuario (USS)	4.2/5	Positivo	La mayoría de los usuarios reporta satisfacción con el producto.
Net Promoter Score (NPS)	45	Adecuado	Hay más promotores que detractores, aunque hay margen de mejora.
Libertad de Riesgo			
Nivel de integridad del sistema	82	Medio	Identifica riesgos de integridad que deben ser mitigados.
Densidad de vulnerabilidades en dependencias (DVD)	0.04	Bueno	Unas pocas dependencias con vulnerabilidades moderadas, requieren actualización.
Cobertura de Contexto			
Complejidad de cobertura de pruebas (CCP)	78 %	Adecuado	La mayor parte de los módulos está cubierta por pruebas automatizadas.
Índice de accesibilidad (IA)	1	Bueno	El sistema es accesible para la mayoría de usuarios.

Cuadro 56: Trazabilidad de resultados a métricas de calidad definidas en la Sección 5

Métrica de calidad	Valor obtenido	Estado	Interpretación
Eficiencia de rendimiento			
Tiempo medio por petición	120 ms	Bueno	Respuesta media de endpoints aceptable para el uso esperado.
Cantidad de recursos usados	35 %	Adecuado	El uso de recursos se mantiene dentro de límites controlados.
Seguridad			
Nivel de integridad del sistema	82	Medio	Existen amenazas de integridad que deben mitigarse.
Nivel de confidencialidad de datos	88	Medio	La confidencialidad es aceptable, pero requiere refuerzos.
Índice de riesgo frontend (IRF)	0.04	Positivo	El puntaje bajo de riesgo frontend indica un nivel elevado de buenas prácticas.
Mantenibilidad			
Porcentaje de código duplicado	12 %	Medio	Código duplicado moderado que debe reducirse para mejorar mantenibilidad.
Deuda técnica promedio por módulo	5.12 h	Medio	Deuda técnica balanceada con módulos que necesitan refactorización.
Complejidad del código	16.2	Aceptable	La complejidad ciclomática promedio es manejable, aunque hay módulos a simplificar.

Cuadro 57: Trazabilidad de resultados de métricas de calidad de producto definidas en la Sección 5

Los resultados obtenidos demuestran que el sistema presenta un alto nivel de cumplimiento funcional y de rendimiento. No obstante, la auditoría permitió identificar vulnerabilidades relacionadas con la seguridad del software, la gestión de dependencias y la configuración del proceso DevSecOps, las cuales deberán ser corregidas antes de la liberación definitiva del sistema. Asimismo, la relación de cada hallazgo con las métricas definidas en la Sección 5 permite cuantificar objetivamente su impacto sobre la calidad del producto y establecer prioridades para su resolución.

SECCIÓN 8. HERRAMIENTAS, TÉCNICAS Y METODOLOGÍAS

Cuadro 58: Herramientas, Técnicas y Metodologías

Herramienta	Uso	Justificación
SonarQube	Análisis estático del código inicial	Herramienta fiable para análisis estático, detecta code smells y problemas estructurales, seguridad con datos cuantitativos útiles para cuantificar la calidad del sistema
Qlty	Análisis estático y evaluación de deuda técnica	Plataforma en la nube que permite evaluar la calidad del código al realizar análisis estático para detectar malas prácticas, vulnerabilidades y bugs. Se destaca por trabajar con varios lenguajes a la vez, y dar un reporte especializado de deuda técnica.
Jacoco	Medición de coberturas de las pruebas	Permitirá evidenciar la cobertura de las pruebas en Java y obtener métricas cuantitativas de la cobertura de las pruebas en el código generado.
npm audit	Análisis de dependencias	Permitirá revisar si los paquetes/bibliotecas están desactualizadas y revisar dependencias en conflicto dentro del frontend.
Junit	Medir funcionalidad en los endpoints del sistema	Permitirá cuantificar y verificar funcionalidad y métricas de tiempo del sistema.
Jmeter	Medir la carga de estrés en el sistema	Permitirá medir la cantidad de esfuerzo que el sistema ejerce y necesita en su funcionamiento.
OWASP ZAP	Medición de la seguridad del sistema	Permitirá revisar problemas de seguridad en los endpoints y encontrar puntos clave de mejora.
Lighthouse	Medir el performance del frontend	Herramienta fiable para medir la usabilidad del sistema y cuantificar la accesibilidad del mismo, separando y mostrando los puntos fuertes y débiles en el frontend.
Cypress	Medir calidad del frontend	Pruebas de interfaz con métricas reales con pruebas en conjunto al funcionamiento del frontend.
Github	Control sobre el proceso QA	Permitirá revisar y tener un espacio controlado donde revisar el sistema y sus cambios.

SECCIÓN 9. CONTROL DEL CÓDIGO

El propósito de esta sección es definir los métodos e instalaciones utilizados para mantener, almacenar, asegurar y documentar las versiones controladas del software identificado durante todas las fases del ciclo de vida del software, cuyo uso apropiado será verificado por SQA. Esto puede implementarse junto con una biblioteca de programas informáticos. Esto puede proporcionarse como parte del Plan SCM. Si es así, se debe hacer una referencia apropiada.

El control del código incluye los siguientes elementos:

1. Identificar, etiquetar y catalogar el software a controlar.
2. Identificar la ubicación física y lógica del software bajo control.
3. Identificar la ubicación, mantenimiento y uso de las copias de respaldo.
4. Distribuir copias del código bajo procedimientos de control de versión.
5. Identificar la documentación afectada por un cambio.
6. Establecer una nueva versión y etiquetado de lanzamientos.
7. Regular el acceso de los usuarios al código.

El proyecto utiliza GitHub como herramienta de control de versiones y gestión de configuración del código fuente. El repositorio remoto del proyecto almacena la totalidad del código fuente del backend Spring Boot, el frontend React, los scripts de despliegue y los documentos de soporte, incluidos los archivos de la carpeta `latex/` y el `README.md`.

La ubicación principal del software bajo control es el repositorio GitHub del proyecto. Los desarrolladores trabajan con copias de trabajo locales sincronizadas con el repositorio remoto. El control de cambios se realiza mediante commits, ramas, pull requests y revisiones de código.

Las copias de respaldo del código se mantienen mediante el propio almacenamiento de GitHub, que conserva el historial de commits y versiones. Además, se recomienda que los desarrolladores mantengan clones locales actualizados y, cuando sea necesario, copias de seguridad adicionales de los archivos de configuración y documentación en el entorno de desarrollo.

La distribución de copias del código se realiza por medio de clones del repositorio y, cuando corresponda, mediante paquetes de entrega formal que incluyan artefactos versionados. La documentación afectada por los cambios en el código se identifica explícitamente en los commits y en los pull requests. Los documentos de diseño, pruebas y calidad vinculados al software se mantienen bajo el mismo control de versiones.

El establecimiento de nuevas versiones se gestiona mediante el etiquetado de commits en Git y la creación de ramas de lanzamiento. Cada versión estable del proyecto debe contar con un tag descriptivo y una descripción de los cambios incorporados en el lanzamiento.

El acceso de los usuarios al código se regula mediante los permisos de repositorio en GitHub. Se emplean controles de acceso para limitar quién puede leer, escribir o aprobar cambios, y se utilizan revisiones de pull requests para verificar que sólo se fusionen cambios autorizados y revisados.

El método de control del código se describe en el Plan SCM del proyecto. SQA realizará evaluaciones continuas del proceso de control del código para verificar que el proceso sea efectivo y cumpla con el Plan SCM. La Sección 3.6 describe más a fondo las actividades de SQA para verificar el proceso de SCM.

SECCIÓN 10. RECOPIACIÓN DE EVIDENCIAS, MANTENIMIENTO Y RETENCIÓN

Identificar la documentación de SQA que se conservará, establecer los métodos e instalaciones que se utilizarán para reunir, salvaguardar y mantener esta documentación, y designar el período de retención.

Las evidencias de pruebas y auditorías de calidad se almacenan en distintos medios según la herramienta empleada. En algunos casos, especialmente cuando la evidencia depende de ejecuciones previas o de entornos externos, es necesario volver a ejecutar la validación para poder consultar o recuperar la información, y únicamente se conserva la configuración o la salida generada por la herramienta.

Evidencia	Formato de guardado	Ubicación	Requiere ejecución
Pruebas JUnit	Scripts en Spring Boot	src/test/	Sí
Pruebas frontend con Cypress	Scripts	frontend/cypress	Sí, por consola o GUI
Auditorías de dependencias	Comando de consola / salida generada	Entorno de ejecución y reporte asociado	Sí, para visualizarse
Pruebas JMeter	CSV, XML o HTML	/jmeter/results	No
OWASP-ZAP	Reporte HTML	Ruta raíz del proyecto	No
Lighthouse	JSON y HTML	frontend/lighthouse-reports	No
SonarQube	Datos almacenados en contenedor de base de datos / Docker	Entorno Docker	Sí, requiere activación
Qlty.sh	Sin formato definido	Nube	No
Videos de ejecuciones de prueba	MP4	https://drive.google.com/drive/u/1/folders/1UJUKMOLDKxPkp8ciLuoG23QkKwSRHRtN	No

Cuadro 59: Evidencias de calidad y su mecanismo de conservación.

Evidencias fotográficas

Se adjuntan a continuación las evidencias fotográficas correspondientes a auditorías y reportes de calidad generados durante la ejecución de las actividades de SQA.

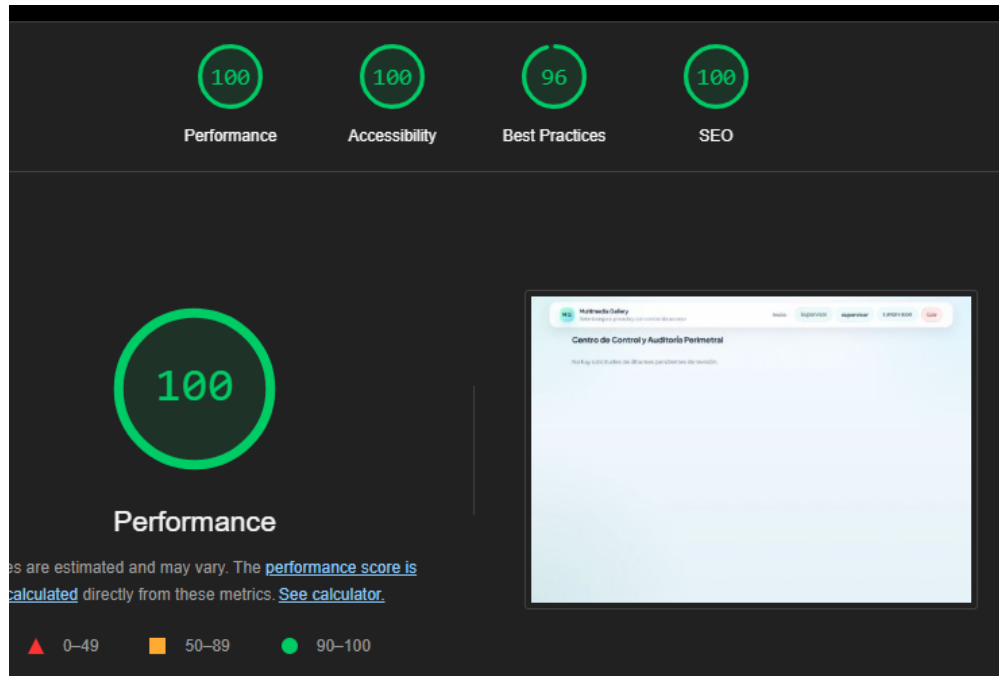


Figura 4: Reporte de Lighthouse

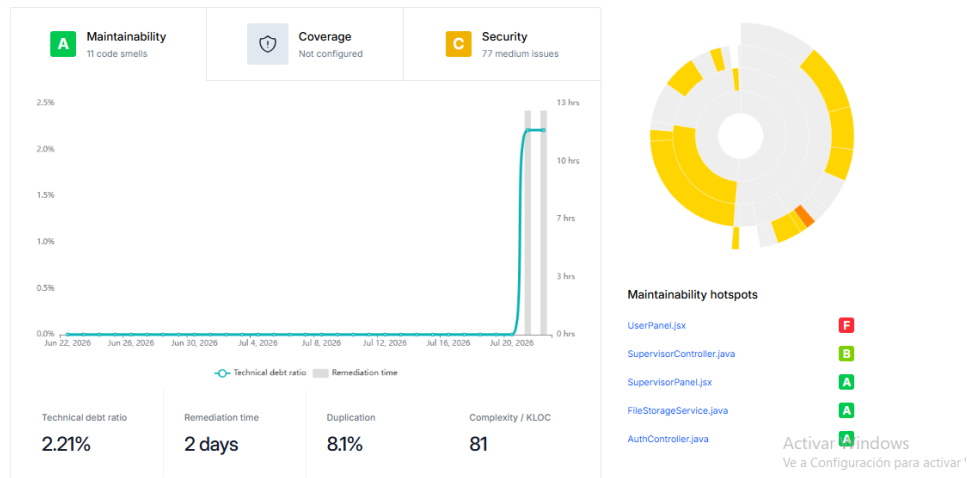


Figura 5: Dashboard de Code Climate

```
added 501 packages, and audited 502 packages in 42s

63 packages are looking for funding
  run `npm fund` for details

20 vulnerabilities (7 low, 6 moderate, 7 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues (including breaking changes), run:
  npm audit fix --force
```

Figura 6: Evidencia de auditoría de dependencias

SECCIÓN 11. PREPARACIÓN DEL PERSONAL

Identificar las actividades de capacitación necesarias para cumplir con los requisitos del Plan SQA.

La Tabla 11-1 proporciona una matriz que identifica las habilidades requeridas para realizar las tareas de SQA para implementar este Plan SQA del proyecto. El cronograma de capacitación será compatible con el cronograma del proyecto. En algunos casos, la capacitación se llevará a cabo como capacitación en el puesto de trabajo (OJT).

Cuadro 60: Matriz de Capacitación SQA

TAREA	REQUISITOS DE HABILIDADES	TIPO	FUENTE
Revisiones de Código	Lenguaje fuente, Revisiones por Pares	Aula/OJT	SEPO, Proceso y Taller de Revisión por Pares
Revisiones de Documentación	Estándares y pautas de desarrollo y documentación de software, Revisiones por Pares	Aula/OJT	SEPO, Proceso y Taller de Revisión por Pares
Auditorías de Procesos	Procesos del Ciclo de Vida del Desarrollo de Software, Técnicas de Auditoría	Aula/OJT	MIL-STD-498, IEEE/EIA 12207
Pruebas	Metodologías de Pruebas	OJT	
Gestión de SQA	Gestión de Proyectos	Aula/OJT	SEPO, Curso de Gestión de Proyectos de Software (SPM)
Métricas	Recopilación y Análisis de Datos	Aula/OJT	SEPO, Curso SPM
Reporte de problemas y acción correctiva	Gestión de Configuración	Aula/OJT	SEPO, Capacitación de Practicante de CM
Herramientas	Capacitación proporcionada por el proveedor	Aula/OJT	Proveedor
Control de Código, Soportes y Proveedores	Gestión de Configuración	Aula/OJT	SEPO, Capacitación de Practicante de CM
Gestión de Riesgos y Análisis	Proceso de Gestión de Riesgos	Aula/OJT	SEPO, Curso SPM
Gestión de Software	Proceso de Gestión de Software	Aula/OJT	SEPO, Capacitación Software Management for Everyone (SME), Curso SPM

SECCIÓN 12. GESTIÓN DE RIESGOS

La gestión de riesgos tiene como objetivo identificar los posibles eventos que pueden afectar la calidad del software o el desarrollo del proceso de pruebas, estableciendo acciones preventivas y correctivas para minimizar su impacto.

Riesgos del Producto

ID	Riesgo	Probabilidad	Impacto	Nivel	Estrategia de Mitigación
RP-01	Acceso no autorizado por vulnerabilidades en JWT o RBAC	Media	Muy Alto	Crítico	Validar autenticación, autorización y expiración de tokens mediante pruebas de seguridad.
RP-02	Ataques XSS mediante contenido enviado por el usuario	Media	Alto	Alto	Aplicar sanitización con OWASP Java HTML Sanitizer y realizar pruebas de penetración.
RP-03	Ataques CSRF sobre operaciones protegidas	Baja	Alto	Medio	Implementar y verificar el funcionamiento del filtro CSRF con Double Submit Cookie.
RP-04	Pérdida de registros de auditoría	Baja	Alto	Medio	Registrar todas las operaciones críticas mediante AuditLog y respaldar la base de datos periódicamente.
RP-05	Fallos en el almacenamiento seguro de imágenes	Baja	Alto	Medio	Utilizar nombres UUID, directorios Safe y Quarantine y validar permisos de acceso.
RP-06	Errores en la validación de archivos (tipo o tamaño)	Media	Medio	Medio	Verificar tamaño máximo de 50 MB, tipos MIME permitidos y reglas de validación antes del upload.

Riesgos del Proyecto de Pruebas

ID	Riesgo	Probabilidad	Impacto	Nivel	Estrategia de Mitigación
RPR-01	Retrasos en el cronograma de pruebas	Media	Alto	Alto	Planificar las actividades por Sprint y realizar reuniones de seguimiento semanales.
RPR-02	Casos de prueba incompletos	Media	Alto	Alto	Elaborar una matriz de trazabilidad entre requisitos y casos de prueba antes de iniciar la ejecución.
RPR-03	Cambios frecuentes en el código durante las pruebas	Alta	Medio	Alto	Utilizar GitHub con ramas de desarrollo y control de versiones para gestionar cambios.
RPR-04	Indisponibilidad de PostgreSQL o ClamAV	Baja	Alto	Medio	Mantener un entorno de pruebas estable y verificar los servicios antes de ejecutar las pruebas.
RPR-05	Baja cobertura de pruebas automatizadas	Media	Medio	Medio	Implementar pruebas unitarias con JUnit y pruebas de API mediante Postman.
RPR-06	Defectos no registrados correctamente	Baja	Medio	Bajo	Mantener un registro centralizado de incidencias con evidencia, prioridad y estado.

Plan de Respuesta a los Riesgos

Para minimizar el impacto de los riesgos identificados durante el proceso de aseguramiento de la calidad, se implementarán las siguientes acciones utilizando las herramientas definidas en el proyecto:

- Ejecutar análisis estáticos del código mediante SonarQube para identificar vulnerabilidades, code smells, errores potenciales y problemas de mantenibilidad antes de iniciar las pruebas dinámicas.
- Medir la cobertura de las pruebas utilizando JaCoCo, con el fin de identificar componentes del sistema que no han sido suficientemente validados y mejorar la confiabilidad del software.

- Analizar las dependencias del proyecto mediante npm audit para detectar bibliotecas vulnerables o desactualizadas y aplicar las actualizaciones o correcciones necesarias.
- Ejecutar pruebas unitarias con JUnit para validar el correcto funcionamiento de los componentes críticos del backend, asegurando que la lógica de negocio opere de acuerdo con los requisitos establecidos.
- Realizar pruebas de seguridad utilizando OWASP ZAP, con el objetivo de identificar vulnerabilidades en la aplicación web, como problemas de autenticación, control de acceso, inyección de código o configuraciones inseguras.
- Evaluar el rendimiento, la accesibilidad y las buenas prácticas del frontend mediante Lighthouse, identificando oportunidades de mejora que contribuyan a una mejor experiencia de usuario.
- Documentar todos los hallazgos, defectos y vulnerabilidades detectadas durante el proceso de pruebas, verificando posteriormente las correcciones mediante pruebas de regresión para asegurar que los cambios implementados no introduzcan nuevos errores.

APÉNDICE A. LISTA DE ACRÓNIMOS

AI - Action Item	PDR - Preliminary Design Review
CDR - Critical Design Review	PP&O - Project Planning and Oversight
CMM - Capability Maturity Model	PRR - Product Readiness Review
CMU - Carnegie-Mellon University	SCM - Software Configuration Management
CRLCMP - Computer Resource Life Cycle Management Plan	SDD - Software Design Document
CI - Configuration Item	SDF - Software Development File
DBDD - Database Design Description	SDP - Software Development Plan
DCR - Document Change Request	SDR - System Design Review
DID - Data Item Description	SEI - Software Engineering Institute
EIA - Electronic Industries Association	SEPO - Systems Engineering Process Office
FCA - Functional Configuration Audit	SME - Software Management for Everyone
FQR - Formal Qualification Review	SPAWAR - Space and Naval Warfare
HB - Handbook	SPI - Software Process Improvement
HWCI - Hardware Configuration Item	SQA - Software Quality Assurance
IDD - Interface Design Description	SRR - System Requirements Review
IEEE - Institute of Electrical and Electronics Engineers	SRS - Software Requirements Specification
IRS - Interface Requirements Specification	SSC - SPAWAR Systems Center
IV&V - Independent Verification and Validation	SSDD - System/Subsystem Design Description
KPA - Key Process Area	SSR - Software Specification Review
MIL - Military	SSS - System/Subsystem Specification
NDS - Non-Developmental Software	STD - Standard
OCD - Operational Concept Document	STR - Software Trouble Report
OJT - On-the-Job	SU - Software Unit
PCA - Physical Configuration Audit	SVD - Software Version Description
P/CR - Problem/Change Report	TRR - Test Readiness Review
	UDF - Unit Development Folder

APÉNDICE B. LISTAS DE VERIFICACIÓN DE AUDITORÍA DE PROCESOS DE INGENIERÍA DE SOFTWARE GENERALES

Cuadro 63: Lista de Verificación de Auditoría del Proceso de Planificación del Proyecto

Proyecto: Fecha: Preparado por:
Procedimientos: ---- Los Planes y compromisos del Proyecto existen y están documentados en el SDP. ---- Los estándares que rigen el proceso de desarrollo de software del proyecto están documentados y reflejados en el SDP. ---- El contenido del SDP refleja la implementación consistente del proceso de software estándar de la organización. ---- El SDP está bajo gestión de configuración. ---- Las actividades de estimación de software se llevan a cabo de acuerdo con el Proceso de Estimación de Software y los resultados están documentados. ---- La base de datos de la organización se utiliza para realizar estimaciones. ---- Los requisitos de software son la base para los planes, productos de trabajo y actividades de software. ---- Existen planes para llevar a cabo la gestión de configuración del software y están documentados en el SDP o en un Plan de Gestión de Configuración de Software (SCM Plan) separado. ---- El Plan SCM está bajo gestión de configuración. ---- Existen planes para llevar a cabo el aseguramiento de la calidad del software y están documentados en el SDP o en un Plan de Aseguramiento de la Calidad del Software (SQA Plan) separado. ---- Existen planes para llevar a cabo las pruebas de integración del software y están documentados en un Plan de Pruebas de Software (STP). ---- Existen planes para llevar a cabo las pruebas del sistema y están documentados en un STP. ---- El STP está bajo gestión de configuración. ---- Existen planes para llevar a cabo la transición del software y están documentados en un Plan de Transición de Software o en el Plan de Desarrollo de Software (SDP). ---- Los cronogramas y planes del proyecto se someten a una revisión por pares antes de establecer sus líneas base.

Cuadro 64: Lista de Verificación de Auditoría del Proceso de Análisis de Requisitos del Sistema

Proyecto: Fecha: Preparado por:
Procedimientos: ---- Los participantes correctos están involucrados en el proceso de análisis de requisitos del sistema para identificar todas las necesidades del usuario. ---- Los requisitos se revisan para determinar si son factibles de implementar, están claramente establecidos y son consistentes. ---- Los cambios en los requisitos asignados, los productos de trabajo y las actividades se identifican, revisan y rastrean hasta su cierre. ---- El personal del proyecto involucrado en el proceso de análisis de requisitos del sistema está capacitado en los procedimientos y estándares necesarios aplicables a su área de responsabilidad para hacer el trabajo correctamente. ---- Los compromisos resultantes de los requisitos asignados son negociados y acordados por los grupos afectados. ---- Los compromisos están documentados, revisados, aceptados, aprobados y comunicados. ---- Los requisitos asignados identificados como potencialmente problemáticos se revisan con el grupo responsable de analizar los requisitos y documentos del sistema, y se realizan los cambios necesarios. ---- Se siguen y documentan los procesos prescritos para definir, documentar y asignar requisitos. ---- Existe un proceso de CM para controlar y gestionar la línea base. ---- Los requisitos están documentados, gestionados, controlados y rastreados (preferiblemente mediante una matriz). ---- Los requisitos acordados se abordan en el SDP.

Cuadro 65: Lista de Verificación de Auditoría del Proceso de Análisis de Requisitos de Software

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Requisitos de Software ---- Los requisitos de software están documentados en una Especificación de Requisitos de Software (SRS) u otro formato aprobado y se incluyen en una matriz de trazabilidad. ---- La SRS se mantiene bajo gestión de configuración. ---- Los cambios en la SRS se someten a una revisión por pares antes de incorporarse a la línea base de requisitos. ---- Asegurar que la revisión por pares valide la capacidad de prueba de los requisitos y que se establezcan métricas apropiadas para validar los requisitos de rendimiento medibles. ---- Los planes, productos de trabajo y actividades de desarrollo de software se cambian para ser consistentes con los cambios en los requisitos de software. ---- Las técnicas de análisis de requisitos de software son consistentes con el SDP. ---- Las herramientas automatizadas adquiridas para gestionar los informes de problemas de requisitos de software y las solicitudes de cambio se utilizan correctamente. ---- El grupo de ingeniería de software está capacitado para realizar actividades de gestión de requisitos. ---- Se realizan mediciones y se utilizan para determinar el estado de la gestión de requisitos. Parte 2. Requisitos de Interfaz ---- Los requisitos de interfaz están documentados en una Especificación de Requisitos de Interfaz (IRS) u otro formato aprobado. ---- La IRS se mantiene bajo gestión de configuración. ---- Los cambios en la IRS se someten a una revisión por pares antes de incorporarse a la línea base de requisitos. ---- Los planes, productos de trabajo y actividades de desarrollo de software se cambian para ser consistentes con los cambios en los requisitos de interfaz. ---- Se utilizan herramientas automatizadas para gestionar los informes de problemas de requisitos de interfaz y las solicitudes de cambio. ---- El grupo de ingeniería de software está capacitado para realizar actividades de gestión de requisitos.

Cuadro 66: Lista de Verificación de Auditoría del Proceso de Implementación y Pruebas Unitarias de Software

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Implementación de Software ---- El código y la matriz de trazabilidad se preparan y se mantienen actualizados y consistentes en función de los cambios aprobados en los requisitos de software. ---- Las revisiones del código (revisión por pares) evalúan el cumplimiento del código con el diseño aprobado, identifican defectos en el código y se evalúan y reportan alternativas. ---- Las revisiones del código se llevan a cabo de acuerdo con el Proceso de Revisión por Pares. ---- Los cambios en el código se identifican, revisan y rastrean hasta su cierre. ---- El código se mantiene bajo gestión de configuración. ---- Los cambios en el código se someten a una revisión por pares antes de incorporarse a la línea base de software. ---- La codificación del software es consistente con la metodología de codificación aprobada en el Plan de Desarrollo de Software (SDP). ---- El método, como la Carpeta de Desarrollo de Software o la Carpeta de Desarrollo de Unidades, utilizado para rastrear y documentar el desarrollo/mantenimiento de una unidad de software está implementado y se mantiene actualizado. Parte 2. Pruebas Unitarias ---- Las pruebas unitarias de software se llevan a cabo de acuerdo con los estándares y procedimientos aprobados descritos en el SDP. ---- Asegurar que los criterios de aprobación para las pruebas unitarias estén documentados y que se haya registrado el cumplimiento. ---- Los resultados de las pruebas unitarias se documentan en la Carpeta de Desarrollo de Software o en la Carpeta de Desarrollo de Unidades.

Cuadro 67: Lista de Verificación de Auditoría del Proceso de Integración y Pruebas Unitarias

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Identificación de Configuración ____ Los CI se integran bajo control de cambios. Si no, vaya a la Parte 2; de lo contrario, continúe. ____ Los CI integrados en el sistema se obtuvieron de un representante autorizado de Gestión de Configuración de acuerdo con el SDP. ____ Las versiones base de cada CI se integran en el sistema. ____ Todos los componentes de CI de la integración de software están bajo control de cambios de acuerdo con el Plan SCM. ____ Si es así, describa cómo se identifican. Parte 2. Proceso de Integración ____ Existe un plan para la integración de los CI. ____ El plan especifica el orden y el cronograma en el que se integran los CI. ____ Los CI se integran de acuerdo con el cronograma y en el orden especificado. ____ El plan de integración especifica qué versión de cada CSC y CSU se va a integrar. ____ Se integra la versión correcta. ____ Los CI integrados han completado las pruebas unitarias. ____ Se han completado las correcciones requeridas. ____ Los CI han sido reprobados. ____ Los procedimientos de prueba están definidos para la integración de CI. ____ Se siguen los procedimientos definidos. ____ Los casos de prueba están definidos. ____ Se siguen los casos de prueba definidos. ____ Los criterios de aprobación/reprobación de las pruebas están definidos. ____ Se siguen los criterios de aprobación/reprobación definidos. ____ Los resultados de las pruebas se documentan en las Carpetas de Desarrollo de Unidades.

Cuadro 68: Lista de Verificación de Auditoría del Proceso de Pruebas de Calificación de CI

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Plan de Pruebas ___ Existe un plan de pruebas y descripciones de pruebas aprobados. ___ El plan y las descripciones utilizados están bajo control de gestión de configuración. ___ Se utiliza la última versión del plan y la descripción. Parte 2. Proceso de Pruebas ___ El software del sistema se recibió de una fuente autorizada de gestión de configuración. ___ El entorno de prueba, incluidos los requisitos de hardware y software, se configura según lo requiere el plan de pruebas. ___ Las pruebas se realizaron en el orden correcto. ___ Se ejecutó cada caso de prueba en la descripción de la prueba. ___ El sistema se prueba después de cada integración de CI. ___ Los criterios de aprobación de las pruebas están documentados y se registra el cumplimiento. ___ Los resultados de las pruebas se registran en un informe de pruebas. ___ Si es así, describa la información que se registra y dónde se registra. ___ Los CI se vuelven a probar después de la integración para asegurar que aún cumplen con sus requisitos sin interferencia del resto del sistema. ___ El sistema que resulta de la integración de cada CI se coloca bajo control de gestión de configuración. ___ Si es así, describa cómo se identifica. Parte 3. Reporte de Problemas ___ Las discrepancias encontradas se ingresan en un sistema de gestión de configuración P/CR o STR para el control de cambios. Si no, describa dónde se registran. ___ Las entradas se completan en el momento en que se encuentran las discrepancias. ___ El número de referencia del P/CR o STR se mantiene en el archivo de pruebas. Si no, describa dónde se guarda. ___ Los P/CR o STR se escriben cuando se encuentran problemas en el entorno de prueba, el plan de pruebas, las descripciones de las pruebas o los casos de prueba. ___ Estos P/CR o STR se envían a través del mismo proceso de control de cambios que las discrepancias de software. Parte 4. Modificaciones ___ ¿Se realizan modificaciones o correcciones en el entorno de prueba durante las pruebas? ___ Si es así, ¿se aprobaron las modificaciones a través del proceso de control de cambios antes de la implementación? ___ ¿Qué documentación de las modificaciones existe? ___ ¿Cuáles son los cambios? ___ ¿Se realizan modificaciones o correcciones en las descripciones de las pruebas durante las pruebas?

Cuadro 69: Lista de Verificación de Auditoría del Proceso de Entrega de Productos Finales

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Auditorías ___ Se realizó una auditoría funcional, si era requerida. ___ Se realizó una auditoría física, si era requerida. Parte 2. Generación del Paquete ___ El software se genera a partir de la biblioteca de software de acuerdo con el SDP. ___ Si es así, el software es la última versión del software en la biblioteca de software. ___ Si no, ¿por qué no? ___ La documentación se genera a partir de los originales controlados por el personal de Gestión de Configuración según lo requiere el Plan SCM. Parte 3. Paquete de Entrega ___ El soporte del software está etiquetado correctamente, mostrando como mínimo el nombre del software, la fecha de lanzamiento y el número de versión correcto. La lista de verificación en la Figura B-12 contiene más detalles sobre el etiquetado de soportes. ___ El Documento de Descripción de la Versión está con el soporte del software. ___ Si es así, el Documento de Descripción de la Versión es la versión correcta para el software. ___ El Documento de Descripción de la Versión ha sido inspeccionado formalmente. ___ El Manual del Usuario está con el soporte del software. ___ Si es así, el Manual del Usuario es la versión correcta para el software. ___ El Manual del Usuario ha sido inspeccionado formalmente. Parte 4. Lista de Distribución de Soportes ___ Existe una lista de distribución para el entregable. ___ Si es así, está completa, todas las organizaciones enumeradas, todas las direcciones correctas y actuales. ___ Si es así, ¿hay alguna organización enumerada que no necesite recibir el entregable? ___ ¿El entregable está clasificado? ___ Si es así, el personal en la lista de distribución tiene la autorización y necesidad de conocer requeridas. Parte 5. Empaquetado ___ El material de empaquetado es adecuado para el contenido y el método de transmisión utilizado. ___ ¿El paquete contiene una carta de transmisión firmada? ___ Si es así, la información de la transmisión es correcta. ___ Todo el contenido está listado en la transmisión contenida en el paquete. ___ ¿El paquete incluye un formulario de acuse de recibo? Parte 6. Notificación de Problemas ___ Existe un método especificado para que la organización receptora notifique al remitente sobre problemas y deficiencias en el paquete.

Cuadro 70: Lista de Verificación de Auditoría del Proceso de Certificación de Soportes

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Producción de Soportes ---- El soporte que contiene el código fuente y el soporte que contiene el código objeto que se entregan se corresponden entre sí. ---- Existe un proceso documentado que se utiliza para implementar la producción de soportes a partir de la biblioteca de software. ---- Si no, se está creando un plan o se está documentando una razón por la que no se necesita uno. Vaya a la Parte 2. ---- El plan se siguió en la producción de soportes. ---- El software se creó a partir de los archivos correctos en la biblioteca de software por personal de CM. ---- Los documentos se crearon a partir de copias maestras aprobadas por personal de CM. Parte 2. Etiquetado de Soportes ---- Existe un estándar documentado que se sigue en el etiquetado de los soportes. ---- Si es así, describa el método estándar utilizado para identificar el producto, la versión y el Número de Identificación de Configuración. ---- El soporte está claramente etiquetado. ---- La etiqueta contiene toda la información requerida (producto, versión y Número de Identificación de Configuración). ---- Si el software está clasificado, el soporte refleja claramente la clasificación correcta. ---- El documento de software está claramente etiquetado con el producto, CIN y número de versión, si corresponde. Parte 3. Contenidos del Soporte ---- Existe un listado del contenido en el soporte. ---- Si es así, describa dónde se encuentra el listado. ---- El soporte contiene el contenido especificado en el listado. ---- El contenido del soporte coincide con la información de la etiqueta, es decir, ¿es la versión correcta para la plataforma de hardware correcta? ---- Los documentos contienen todas las páginas de cambio requeridas para esta versión de los documentos.

Cuadro 71: Lista de Verificación de Auditoría del Proceso de Certificación de Software No Entregable

Proyecto: Fecha: Preparado por:
Procedimientos: ---- El software entregable depende del software no entregable. ---- Si es así, se ha dispuesto que el adquiriente tenga o pueda obtener el software no entregable. ---- El software no entregado cumple con el uso previsto. ---- El software no entregado se coloca bajo gestión de configuración.

Cuadro 72: Lista de Verificación de Auditoría del Proceso de Almacenamiento y Manipulación

Proyecto: Fecha: Preparado por:
Procedimientos: ---- Los documentos y soportes se almacenan de acuerdo con el procedimiento de la Biblioteca de Desarrollo de Software. ---- Las áreas de almacenamiento de productos en papel están libres de efectos ambientales adversos (alta humedad, fuerzas magnéticas, calor y polvo). ---- Las áreas de almacenamiento de productos en soportes están libres de efectos ambientales adversos (alta humedad, fuerzas magnéticas, calor y polvo). ---- Los contenedores de almacenamiento para material clasificado son apropiados para el nivel de material clasificado.

Cuadro 73: Lista de Verificación de Auditoría del Proceso de Desviaciones y Exenciones

Proyecto: Fecha: Preparado por:
Procedimientos: ---- Las desviaciones y exenciones se preparan de acuerdo con los procedimientos documentados en el Plan SCM del proyecto. ---- Las desviaciones y exenciones son revisadas y aprobadas por el personal apropiado de acuerdo con el Plan SCM del proyecto. ---- Los registros de desviaciones y exenciones se mantienen y reflejan la configuración de desarrollo actual.

Cuadro 74: Lista de Verificación de Auditoría del Proceso de Gestión de Configuración de Software

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Plan SCM ____ El proyecto sigue la política organizacional para implementar SCM. ____ Existe el grupo responsable de coordinar e implementar SCM para el proyecto. ____ Se utiliza un Plan SCM documentado y aprobado como base para realizar las actividades de SCM. ____ El control de configuración de los cambios en los documentos y software de línea base se gestiona de acuerdo con el Plan SCM. ____ Se establece un sistema de biblioteca de gestión de configuración como repositorio de la línea base de software. ____ La biblioteca de CM es el único lugar de almacenamiento de la versión base de todo el software. ____ El acceso a los productos de software en la biblioteca de CM se realiza de acuerdo con los procedimientos de Control de la Biblioteca. ____ Los productos de trabajo de software que se colocarán bajo SCM se identifican de acuerdo con el Plan SCM. ____ Existe la Junta de Control de Cambios Local (LCCB) e implementa los procedimientos de la LCCB. ____ Las solicitudes de cambio y los informes de problemas para todos los elementos de configuración se manejan de acuerdo con el procedimiento de PCR. ____ Los cambios en las líneas base se controlan de acuerdo con el Plan SCM, el procedimiento de la LCCB y el procedimiento de PCR. ____ Los productos de la biblioteca de línea base de software se crean y su liberación se controla de acuerdo con los procedimientos de Control de la Biblioteca. ____ Los informes de contabilidad del estado de configuración se preparan de acuerdo con el Plan SCM. Parte 2. Identificación de Configuración ____ Se pueden identificar las Líneas Base de Productos y la Biblioteca de Desarrollo. ____ Si es así, describa el método utilizado para identificar las Líneas Base y la Biblioteca de Desarrollo. ____ Enumere los documentos que componen estas Líneas Base y la Biblioteca de Desarrollo. ____ Se puede identificar la documentación y los soportes de almacenamiento informático que contienen código, documentación o ambos. ____ Si es así, describa el método utilizado para identificar la documentación y los soportes de almacenamiento informático y enumere los documentos que se colocan bajo control de configuración. ____ Se puede identificar cada CI y sus componentes correspondientes. ____ Si es así, describa el método utilizado para identificarlos. ____ Se utiliza un método para identificar el nombre, la versión, el lanza-

Cuadro 75: Lista de Verificación de Auditoría del Proceso de Control de la Biblioteca de Desarrollo de Software

Proyecto: Fecha: Preparado por:
Procedimientos: Parte 1. Establecimiento del Entorno y Control de la SDL ____ La SDL está establecida y existen procedimientos para gobernar su operación. ____ La SDL proporciona un medio positivo para reconocer los elementos relacionados (es decir, aquellas versiones que constituyen una línea base particular y proteger el software contra la destrucción o modificación no autorizada). ____ La documentación y los materiales del programa informático aprobados por la LCCB se colocan bajo control de la biblioteca. ____ Todo el software, herramientas y documentación relevantes para el desarrollo de software se colocan bajo control de la biblioteca. ____ Existen procedimientos/estándares publicados para la SDL. ____ Los procedimientos de la SDL incluyen la identificación de las personas/organización responsables de recibir, almacenar, controlar y difundir los materiales de la biblioteca. ____ El acceso a la SDL está limitado al personal autorizado. ____ Si es así, describa los procedimientos utilizados para limitar el acceso. ____ Existen salvaguardas para asegurar que no se realicen alteraciones no autorizadas al material controlado. ____ Si es así, describa esas salvaguardas. ____ Describa o enumere los materiales a controlar. ____ Describa cómo se aprueban y colocan bajo control los materiales. ____ Describa cómo se manejan los cambios en la parte del software y cómo se identifican los cambios en las líneas de código. ____ La SDL contiene copias maestras de cada CI bajo control de la Biblioteca de Programas Informáticos. ____ Se realiza una copia de seguridad periódica del software para evitar la pérdida de información debido a cualquier falla del Sistema de Biblioteca de Desarrollo. ____ Si es así, describa el procedimiento de copia de seguridad y la frecuencia de las copias de seguridad. Parte 2. Garantía de Validez del Material Controlado ____ Las duplicaciones de copias maestras controladas y probadas se verifican antes de la entrega como copias exactas. ____ Todos los productos de software entregables que se duplican de copias maestras controladas y probadas se comparan con esa copia maestra para asegurar la duplicación exacta. ____ Describa cómo se asignan los números de identificación y los códigos de revisión a los documentos y software controlados. ____ Describa cómo se registran las liberaciones de materiales controlados. ____ Enumere las personas u organización responsables de la garantía de la validez del soporte de software. ____ Existe un procedimiento de liberación formal. Si es así, ¿describalo?

Cuadro 76: Lista de Verificación de Auditoría del Proceso de Software No Desarrollado

Proyecto: Fecha: Preparado por:
Procedimientos: ---- El software no desarrollado cumple con las funciones previstas. ---- El software no desarrollado se coloca bajo CM interno. ---- Las disposiciones de derechos de datos y licencias son consistentes con el SDP.

SOLICITUD DE CAMBIO DE DOCUMENTO (DCR)

Título del Documento: [Nombre del Proyecto] Plan de Aseguramiento de la Calidad del Software	Número de Seguimiento:
Nombre de la Organización Solicitante:	
Contacto de la Organización:	Teléfono:
Dirección Postal:	
Título Corto:	Fecha:
Ubicación del Cambio: (usar número de sección, número de figura, número de tabla, etc.)	
Cambio Propuesto:	
Razón del Cambio:	
Nota: Para que [Nombre del Proyecto] tome las medidas adecuadas en una solicitud de cambio, proporcione una descripción clara del cambio recomendado junto con la justificación correspondiente. Enviar a: Comandante, Space and Naval Warfare Systems Center, Code 2*XX*, 53560 Hull Street, San Diego, CA 92152-5001. Fax: <i>agregar número de fax apropiado</i> . Correo electrónico: <i>agregar correo electrónico apropiado</i> .	